

HCMC UNIVERSITY OF TECHNOLOGY AND ENGINEERING
FACULTY OF ADVANCED EDUCATION



HCMUTE

TOPIC

Design and implementation smart indoor environment monitoring and climate control system

Internet of Things Foundations and Application final essay

COURSE CODE: ITFA336064E

CLASS COURSE: ITFA336064E_04FIE

CLASS PERIOD: Wednesday & Time 6-10

INSTRUCTOR: Dr. Thien Huynh-The

STUDENT MEMBERS:

Lê Thành Tài – 24119080

Lê Công Quốc Anh – 24119009

Nguyễn Phi Lanh – 24119052

Phan Anh Khoa – 24119048

Huỳnh Anh Tuấn – 24119096

Ho Chi Minh City, September, 2026

Acknowledgements

We would like to express our sincere and deepest gratitude to our supervisor, **Dr. Thien Huynh-The**, for his dedicated guidance, support and valuable knowledge throughout our studies and the implementation of the project “Design and Implementation of a Smart Indoor Environment Monitoring and Climate Control System”.

We also thank the Board of the Faculty, the lecturers of the Faculty of Special Training and Ho Chi Minh City University of Technology and Education (HCMUTE) for providing the facilities and the learning environment that allowed our team to complete this project. Despite our efforts in hardware design, embedded firmware, the real-time time-series database and the integration of an artificial-intelligence model, the project inevitably has shortcomings because of limited time and practical experience. We would be very grateful for any comments and suggestions from our teachers.

Thank you very much!

Motivation for Choosing the Topic

In the context of rapid urbanisation and a steadily rising quality of life, the indoor living environment, and the kitchen in particular, has a direct influence on respiratory health and personal safety. Cooking generates large amounts of smoke, fine particulate matter ($\text{PM}_{2.5}$, PM_{10}), volatile organic compounds (VOCs) and other harmful gases, usually together with high temperature and humidity.

Conventional extractor fans depend entirely on the user switching them on and off by hand. They are therefore often started too late, when harmful gases have already spread, or are left running and waste energy. In addition, the lack of remote monitoring, time-series data storage and AI-based trend forecasting means that such solutions do not meet the requirements of modern smart buildings.

Motivated by these practical challenges, our team chose the project “**Design and Implementation of a Smart Indoor Environment Monitoring and Climate Control System**”. The goal is a complete IoT system from the edge to the cloud, combining an FSM controller that prevents chattering, the TimescaleDB time-series database, a web dashboard for visual monitoring, and an AI model that forecasts air-quality risk.

Contents

Acknowledgements	i
Motivation for Choosing the Topic	ii
List of Figures	vii
List of Tables	ix
1 Overview and Theoretical Background	1
1.1 Problem Statement	1
1.2 Objectives and Scope	1
1.2.1 End-to-End Architecture	2
1.2.2 Comparison with Conventional Solutions	2
1.2.3 Scope and Limitations	2
1.3 Bill of Materials (BOM)	3
1.4 Hardware Construction Results	4
2 Sensor Calibration & Display	5
2.1 MQ-135 Operating Principle and Burn-in Procedure (12–24h)	5
2.2 Sensor Resistance R_s and Determination of R_0	5
2.2.1 Determining R_0	6
2.3 Temperature and Humidity Compensation with the AHT20	8
2.4 LCD Interface with Three Rotating Screens	8
3 Firmware, MQTT Protocol and Control Logic (FSM)	10
3.1 MQTT Communication Architecture	10
3.1.1 Topic Tree	10
3.1.2 Payload Structure	11

3.1.3	Message Sequence	13
3.2	QoS Analysis and Selection	14
3.3	Last Will and Testament (LWT)	15
3.4	Finite State Machine for Fan Control	17
3.4.1	Inputs, Thresholds and Fan Levels	17
3.4.2	State Logic	18
3.4.3	State Diagram	19
3.4.4	Truth Table	19
3.4.5	Why Two Variables	20
3.4.6	AUTO and MANUAL Modes	20
3.5	Chattering Prevention	21
3.5.1	The Problem	21
3.5.2	Hysteresis	21
3.5.3	Limits of the Current Implementation and Dwell Time	22
3.6	Network Loss Handling	22
3.6.1	Non-Blocking Program Structure	22
3.6.2	Online Mode and Local Autonomous Fallback	24
3.6.3	Reconnection Strategy	25
4	Backend, Database and Alerting System	27
4.1	Overview of Responsibilities and Backend Architecture	27
4.2	Mosquitto MQTT Broker and Access Control	28
4.3	Time-Series Database Schema Design	31
4.3.1	Database Schema Specification	31
4.4	Backend Server Architecture and Ingestion Service	33
4.5	REST API Specification	35
4.6	Discord Webhook Alerting System Integration	37
4.7	AI Time-Series Forecasting Model Integration	39
4.8	Containerized Deployment with Docker Compose	40
4.9	Backend – Dashboard Integration Points	43
5	Real-Time Web Dashboard	45
5.1	Frontend Architecture and Design Philosophy	45
5.1.1	Technology Stack Selection	45

5.1.2	Component Hierarchy and Responsive Layout	46
5.2	Real-Time MQTT Communication over WebSockets	47
5.2.1	Event-Driven WebSocket Protocol	47
5.2.2	Ghost Packet Suppression and Multi-Layer Filtering	48
5.3	Telemetry Widgets and Diagnostic Modals	50
5.3.1	Telemetry Cards and State Indicators	50
5.3.2	WiFi Node Diagnostics Modal	50
5.3.3	Air Quality Index and Health Score Formula	50
5.4	Data Freshness and Dual-Tier Hardware Watchdog	51
5.4.1	Mathematical Formulation of Data Freshness	51
5.4.2	Dual-Tier Watchdog Engine (5s Startup vs. 10s Power Cut)	52
5.5	Actuation Control and Decoupled Siren Logic	54
5.5.1	Bimodal Arbitration: AUTO versus MANUAL	54
5.5.2	Binary Actuation and Alarm Decoupling	54
5.6	Continuous Waveform Visualization and Historical Buffering	58
5.6.1	Chart.js Multi-Axis Stream Integration	58
5.6.2	Dual-Mode Buffering (Live 20-Sample Stream vs. 24-Hour History)	58
5.6.3	System Event Logging and Threshold Configuration Modals	59
5.6.4	Exporting Telemetry to CSV	61
5.7	Testing and Deployment Verification	61
5.7.1	Production Deployment on Vercel	61
6	AI Foundations, Model Testing and Quantitative Experiments	62
6.1	Overview of the Indoor Air Quality (IAQ) Prediction Problem	62
6.2	Design of Features and Structure of the Training Dataset	63
6.3	Mathematical Foundations of Applied Artificial Intelligence Algorithms . .	64
6.3.1	Holt-Winters Exponential Smoothing Model (HWES)	64
6.3.2	Short-Term Memory Network (LSTM)	65
6.4	Model Accuracy Assessment: Relationship Between MAE and RMSE . . .	66
6.5	Quantitative Experimentation and Testing of Industrial Readiness	67
6.5.1	Test 1: Measuring System-Wide Latency (Latency & RTT)	67
6.5.2	Test 2: Evaluating Chattering Phenomena (On/Off Oscillations) Under the Influence of FSM	67

6.5.3	Test 3: Local Autonomy in the Event of Network Fallback and Connection Recovery	68
6.5.4	Test 4: Evaluation of Ventilation Performance and Air Clearance Time According to ASHRAE 62.2	68
6.6	Project Summary and Practical Recommendations	69
7	Future Developments	70
7.1	Multi-Room and Multi-Zone Deployment	70
7.2	Edge AI with a More Powerful Microcontroller and TensorFlow Lite	70
7.3	Completing the Dwell-Time Lock	71
7.4	Industrial-Grade Sensors and Real-World Validation	71
	References	72

List of Figures

1.1	Overall end-to-end block diagram (data direction and protocol between blocks).	2
1.2	Overall circuit schematic of the smart kitchen monitoring node	4
2.1	Voltage-divider circuit of the MQ-135 and the $4.7\text{ k}\Omega/4.7\text{ k}\Omega$ divider protecting the ESP32 ADC.	6
3.1	Topic tree of the node. Green: topics published by the ESP32. Blue: topics subscribed by the ESP32.	11
3.2	Sequence of the MQTT messages between the ESP32, the broker, the backend and the dashboard.	13
3.3	Timeline of an unexpected loss of the node: the will message is published after the keep-alive time.	16
3.4	State diagram of the fan control FSM.	19
3.5	Principle of the hysteresis band (schematic). Fluctuations between 950 and 1000 ppm do not change the state.	22
3.6	Transition between the online mode and the local autonomous fallback mode.	25
4.1	Backend architecture and data flows. Every block inside the dashed box is a Docker container on the AWS EC2 server; the browser and the ESP32 reach it only through ports 443 and 1883.	28
5.1	Desktop cockpit view showing the command bar, central atmosphere reactor, telemetry matrix, and navigation tabs.	46
5.2	Responsive mobile layout captured across 3 vertical viewports for handheld supervision.	47
5.3	WiFi Node Diagnostics Modal providing network performance metrics.	51
5.4	Visual state during hardware disconnect: crimson offline banner, dimmed telemetry cards, and nullified metric values (-).	53

5.5	Turbine Fan actuator control states: Manual OFF, Manual ON (100%), and AUTO FSM Management.	54
5.6	24-Hour Historical Environmental Record view showing 291 cached records, a max gas peak of 9999 ppm, and 9 hazard detections.	59
5.7	System Activity and State Logs view displaying RTT performance (150.8 ms) and AI inferences.	60
5.8	Threshold Tuning calibration modal adhering to schema v1.2.	60

List of Tables

1.1	Comparison of the proposed system with conventional solutions.	3
1.2	Bill of Materials (BOM) of the proposed system.	3
2.1	V_{RL} , R_s , $R_{s,corr}$ and R_0 measured in clean air (repeated measurements, averaged).	7
3.1	MQTT topics used by the ESP32 firmware	11
3.2	MQTT quality of service levels	14
3.3	QoS and retain flag of each topic in the firmware	14
3.4	Parameters of the fan control FSM	17
3.5	Next state of the FSM	20
3.6	Periodic tasks of the firmware	24
3.7	Waiting time of the proposed exponential backoff	26
4.1	MQTT accounts and their ACL permissions	30
4.2	MQTT topics defined by the data contract v1.2	30
4.3	Tables of the <code>iot_kitchen</code> database	31
4.4	Telemetry table schema (<code>telemetry</code>)	32
4.5	Backend REST API endpoints (prefix <code>/api/v1/kitchen</code> ; * requires the <code>X-API-Key</code> header)	36
4.6	Alert rules	38
4.7	Docker Compose services	42
5.1	Data Freshness Classification Spectrum	52
6.1	Distribution of telemetry latency and control loop response time across 30 samples	67
6.2	Comparison of the control stability of physical actuators under amplitude disturbances	68

Chapter 1

Overview and Theoretical Background

1.1 Problem Statement

The kitchen is one of the main sources of indoor air pollution. Cooking releases smoke, oil vapour, carbon dioxide, carbon monoxide and volatile organic compounds. If these pollutants are not removed in time, their concentration rises quickly. This causes discomfort, affects the respiratory system and increases safety risks. In practice, most extractor fans are switched on and off by hand. Users often turn them on too late or forget to turn them off, which wastes energy.

This project therefore builds a monitoring and control system. Sensors continuously measure air quality, temperature and humidity. An ESP32 microcontroller processes the data, shows it on an LCD 1602 display and switches a 12 V extractor fan automatically when the pollution level exceeds a threshold. Because the system is intended as a complete IoT prototype, the hardware must be stable, the sensor data must be trustworthy, and the power stage must be safe for the microcontroller.

1.2 Objectives and Scope

1. **Hardware.** Complete the circuit consisting of the ESP32, the MQ-135 gas sensor, the AHT20 temperature and humidity sensor, a LCD 1602 display and a MOSFET driver for a 12 V fan.
2. **Calibration and data processing.** Do not use raw ADC values. Instead, compute the sensor resistance R_s , determine the reference resistance R_0 , compensate for temperature and humidity, and convert the result to a pollution percentage from 0%

to 100%.

3. **Display and integration.** Program the LCD to rotate through three screens, then mount the whole system in an acrylic (Plexiglas) kitchen chamber for real tests.

1.2.1 End-to-End Architecture

The system consists of the following blocks: sensors (MQ-135, AHT20) → ESP32 micro-controller (processing, LCD display, fan control) → MQTT broker (Mosquitto) → backend and database (storage, alerting, AI) → web dashboard / Telegram. Figure 1.1 shows the direction of data flow and the protocol used between the blocks.

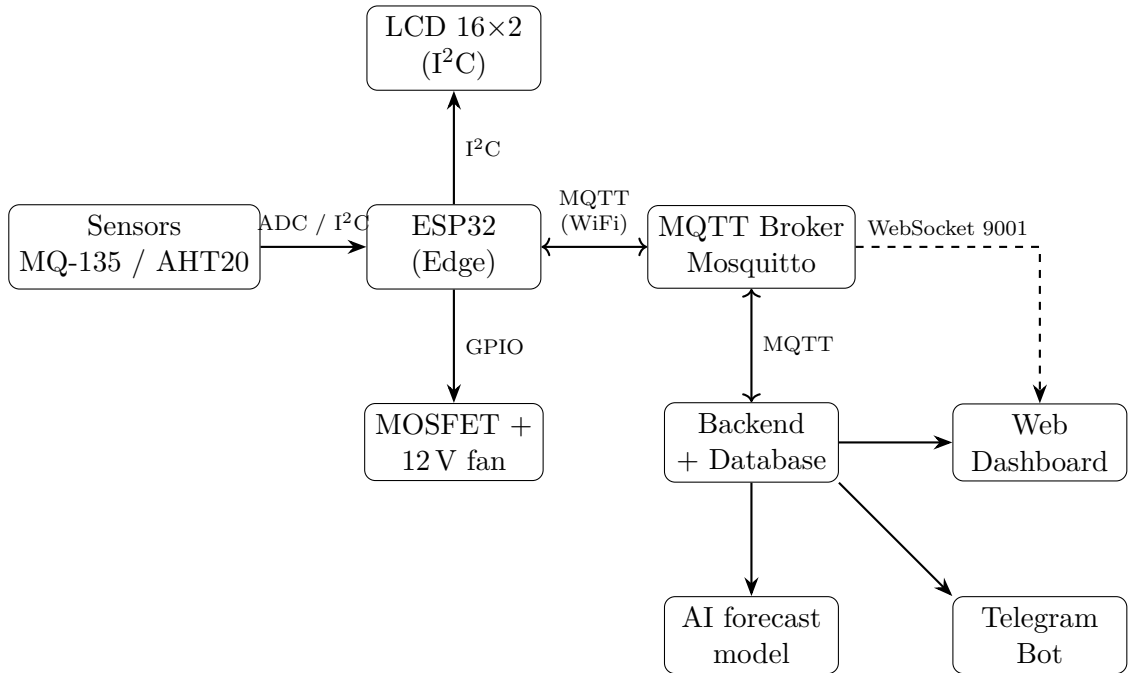


Figure 1.1: Overall end-to-end block diagram (data direction and protocol between blocks).

1.2.2 Comparison with Conventional Solutions

The novelty of the proposed system is the combination of **automatic control**, **remote monitoring** and **AI-based forecasting** in a single low-cost platform.

1.2.3 Scope and Limitations

The prototype consists of a single simulated kitchen chamber, with incense used as the smoke source. It has not been deployed in real kitchens or across multiple rooms, and it has not been calibrated against industrial reference instruments.

Table 1.1: Comparison of the proposed system with conventional solutions.

Criterion	Mechanical extractor fan	Simple smoke detector	Proposed system
Automatic on/off	No	Alarm only	Yes (multivariable FSM)
Temperature/humidity sensing	No	No	Yes (AHT20)
Remote monitoring	No	No	Yes (dashboard, Telegram)
Trend forecasting	No	No	Yes (AI model)
Operation without network	Yes (manual)	Yes	Yes (local fallback)

1.3 Bill of Materials (BOM)

Table 1.2: Bill of Materials (BOM) of the proposed system.

Component	Specification	Qty	Role in the system
ESP32 DevKit	WiFi 2.4 GHz, I ² C, GPIO	1	Central microcontroller
AHT20	I ² C (0x38), $\pm 0.3^{\circ}\text{C}$, $\pm 2\%\text{RH}$	1	Temperature and humidity sensing
MQ-135	SnO ₂ gas sensor, 5 V heater	1	Air-pollution sensing
LCD 1602 + I ² C	I ² C address 0x27	1	Local display
PWM MOSFET Power Control Module		1	Fan speed and switching control
DC fan 12 V		1	Simulated range hood
Power supply 12 V 1 A	DC adapter	1	Main system supply
Resistors	4.7 k Ω	2	Gate pull-down, current limit, ADC divider
Capacitors	10nF, 47 μF	4	Low-pass filter and supply filtering
Phomex	3 mm sheet, 15 $\times$ 10 $\times$ 12 cm	1	Kitchen chamber frame

Rationale for the Main Components

ESP32 the ESP32 has built-in WiFi for MQTT, two cores, a 12-bit ADC and NVS flash for storing R_0 . Moreover, it has a low cost.

MQ-135 the MQ-135 responds to NH₃, benzene, smoke and CO₂, which suits general air-quality assessment.

AHT20: I²C interface and better accuracy than the DHT11; it supplies T and H for compensating the MQ-135.

I²C character LCD: the team uses a standard **LCD 1602** (16×2 characters). It needs only two signal wires (SDA/SCL), shares the bus with the AHT20, is easy to read in a bright kitchen thanks to its backlight, and is cheap and simple to program.

PWM Mosfet: It can easy control Fan speed and switching by PWM of MCU.

1.4 Hardware Construction Results

Figure 1.2 shows the complete schematic of the node.

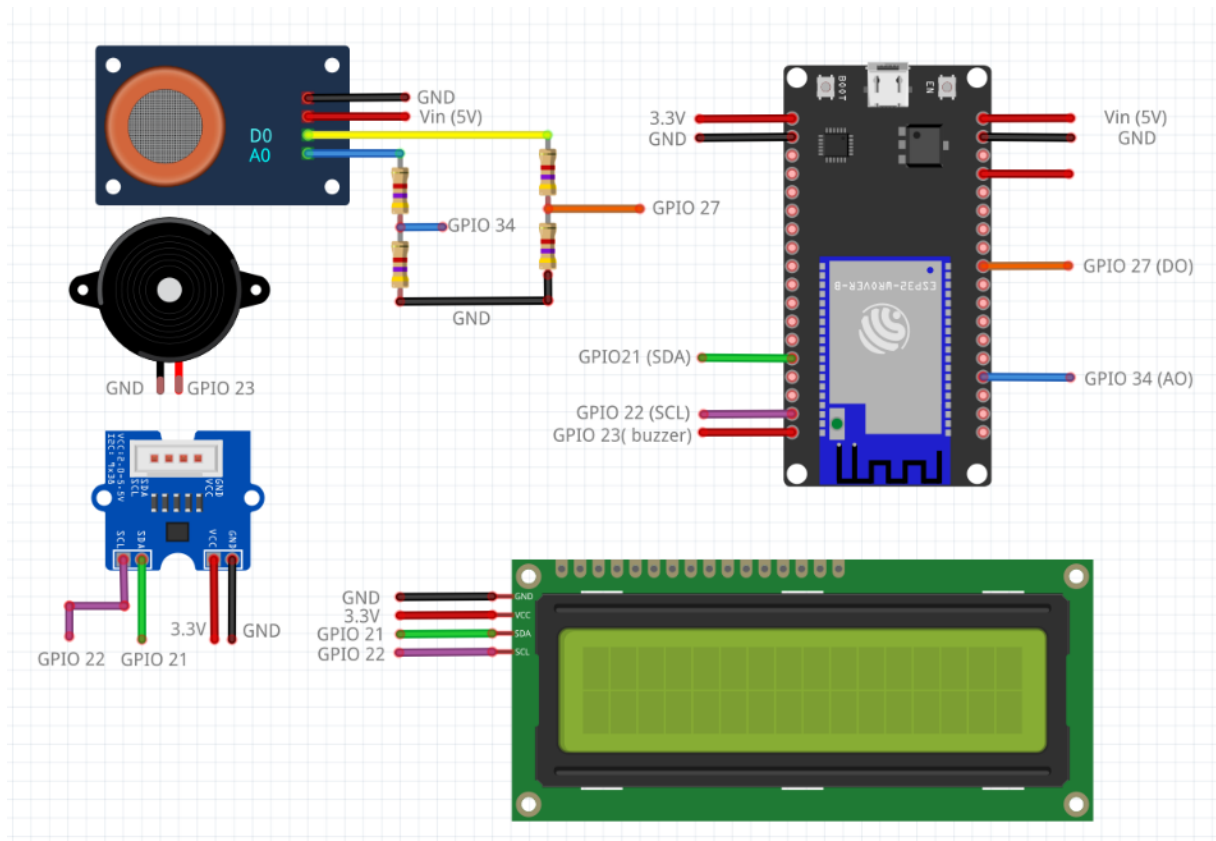


Figure 1.2: Overall circuit schematic of the smart kitchen monitoring node

Assessment: *Mechanics:* the acrylic/foam frame is light and stiff enough to hold the fan, sensors and LCD without deformation, and the ceiling fan mount is easy to service. *Appearance:* the transparent panels let the smoke be observed, and the LCD on the front face is easy to read. *Durability:* the main drawbacks are that foam edges and glued joints are not airtight (some smoke leaks, which slows the build-up of pollution in the chamber), and that acrylic can crack near screw holes. Sealing the joints with tape or silicone would improve repeatability of the smoke experiments.

Chapter 2

Sensor Calibration & Display

2.1 MQ-135 Operating Principle and Burn-in Procedure (12–24 h)

The MQ-135 is a metal-oxide semiconductor gas sensor whose sensing layer is mainly SnO_2 . A heater element keeps the layer at its working temperature. In clean air, oxygen adsorbed on the surface captures electrons from the material, forming a potential barrier, so the sensor resistance is high. When polluting gases (NH_3 , benzene, smoke, CO_2 , etc.) are present, the gas molecules react with the adsorbed oxygen and release electrons back into the material, so the sensor resistance R_s decreases as the gas concentration increases.

A new sensor needs **burn-in**: the heater is powered continuously for 12–24 hours so that the sensing layer reaches a stable state. If this step is skipped, the resistance drifts strongly and the calibrated R_0 is unreliable. In this project the burn-in lasted [24 hours](#) in a well-ventilated room at about [28°C](#), [60–70% RH](#), with the heater supplied continuously at 5 V.

2.2 Sensor Resistance R_s and Determination of R_0

The module connects the MQ-135 in series with a load resistor R_L , forming a voltage divider; the voltage across R_L (V_{RL}) is the analogue output A0. Applying Ohm's law to the divider gives

$$R_s = \frac{V_{CC} - V_{RL}}{V_{RL}} R_L, \quad (2.1)$$

where $V_{CC} = 5.0 \text{ V}$ is the supply of the divider (sensor + R_L), V_{RL} is the measured voltage across R_L , and $R_L = 2 \text{ k}\Omega$ is the load resistor on the module.

Because A0 can reach 5 V while the ESP32 ADC tolerates at most 3.3 V, A0 goes

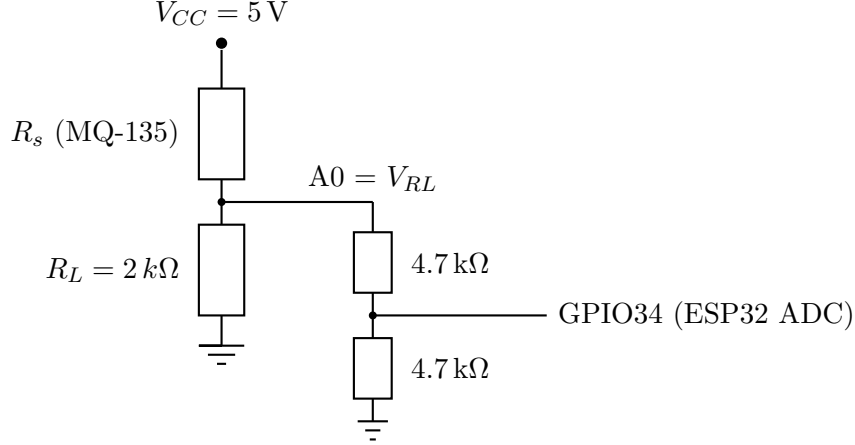


Figure 2.1: Voltage-divider circuit of the MQ-135 and the $4.7\text{k}\Omega/4.7\text{k}\Omega$ divider protecting the ESP32 ADC.

through a divider of $4.7\text{k}\Omega$ in series with $4.7\text{k}\Omega$ to GND (ratio $1/2$). The firmware multiplies by 1.5 to recover the real voltage:

$$V_{RL} = V_{\text{GPIO34}} \times 2. \quad (2.2)$$

To reduce noise, each measurement averages 32 ADC samples read with the ESP32 calibrated-voltage function (`analogReadMilliVolts`). After the fault check of Section ??, V_{RL} always lies in $[0.05, 4.55]\text{ V}$, so the division in Eq. (2.1) is safe.

2.2.1 Determining R_0

R_0 is the resistance of the same sensor in clean air. It is measured outdoors in clean air at 20°C and 65% RH. Because the temperature/humidity correction $K(T, H)$ of Section 2.3 gives $K(20^\circ\text{C}, 65\%) = 0.938$ at exactly these conditions, the measured R_s is first compensated and then divided by the clean-air ratio:

$$R_0 = \frac{R_s / K(20^\circ\text{C}, 65\%) }{3.6} = \frac{R_s}{0.938 \times 3.6}. \quad (2.3)$$

Why 3.6: on the standard R_s/R_0 curve in the MQ-135 datasheet [2], the ratio in clean air at 20°C and 65% RH is approximately 3.6; hence R_0 is the reference resistance used to compare sensors and measurements. Using the same compensated quantity $R_{s,\text{corr}}$ for calibration and for measurement keeps the two consistent: in clean air at $20^\circ\text{C}/65\%$ RH the pollution percentage is exactly 0%.

Example with $V_{RL} = 0.74\text{ V}$: $R_s = \frac{5-0.74}{0.74} \times 2 = 11.51\text{ k}\Omega$ and $R_0 = 33.74/3.6 = 3.19\text{ k}\Omega$. Averaging repeated measurements reduces random error. R_0 is stored in the ESP32 NVS/flash so that recalibration is not needed after a power loss; it is redone only when the sensor is replaced or the operating environment changes.

Table 2.1: V_{RL} , R_s , $R_{s,corr}$ and R_0 measured in clean air (repeated measurements, averaged).

Trial	V_{RL} (V)	R_s (k Ω)	R_0 (k Ω)
1	0.75	11.33	3.35
2	0.79	10.65	3.56
3	0.73	11.69	3.46
4	0.7	12.28	3.63
5	0.71	12.08	3.57
Mean	0.736	11.60	3.51

```
1 #include <Arduino.h>
2
3 #define AO_PIN 4
4 #define VCC 5.0
5 #define RL 2000.0      // 202 = 2 k
6 #define R0 3200.0
7
8 void setup() {
9     Serial.begin(115200);
10    analogReadResolution(12);
11 }
12
13 void loop() {
14     const int N = 50;
15     long sum = 0;
16     for (int i = 0; i < N; i++) {
17         sum += analogRead(AO_PIN);
18         delay(5);
19     }
20     float adc = sum / (float)N;
21     float Vadc = adc * 3.3 / 4095.0;
22     float Vao = Vadc * 2;
23     float Rs = ((VCC - Vao) / Vao) * RL;
24     float ratio = Rs / R0;
25
26     Serial.println("-----");
27     Serial.print("ADC      : ");
28     Serial.println(adc);
29     Serial.print("V_ADC    : ");
30     Serial.print(Vadc, 3);
31     Serial.println(" V");
32     Serial.print("V_AO     : ");
33     Serial.print(Vao, 3);
34     Serial.println(" V");
35     Serial.print("Rs       : ");
36     Serial.print(Rs, 1);
```

```

37 Serial.println(" ohm");
38 Serial.print("Rs/R0      : ");
39 Serial.println(ratio, 3);
40 delay(1000);
41 }

```

Listing 2.1: Saving and loading R_0 with the Preferences library (illustrative).

2.3 Temperature and Humidity Compensation with the AHT20

The sensitivity of the MQ-135 changes with ambient temperature and humidity: the same gas concentration gives a different R_s under different weather conditions. The system therefore reads T and H from the AHT20 (I²C, address 0x38) and computes a correction factor $K(T, H)$ from the datasheet:

$$K(T, H) = \begin{cases} 0.00035 T^2 - 0.02718 T + 1.39538 - 0.0018 (H - 33), & T < 20^\circ\text{C}, \\ -0.003333 T - 0.001923 H + 1.130128, & T \geq 20^\circ\text{C}, \end{cases} \quad (2.4)$$

$$R_{s,\text{corr}} = \frac{R_s}{K(T, H)}. \quad (2.5)$$

Values between the datasheet points are interpolated by the function above (quadratic in T , linear in H); the two branches differ by less than 0.01 at $T = 20^\circ\text{C}$, so the correction is practically continuous. If the AHT20 read fails, K is set to 1 so that the system keeps running, and an AHT20 error flag is raised.

2.4 LCD Interface with Three Rotating Screens

Three screens rotate on the 16×2 character LCD, each shown for 3 seconds: **(1) room temperature** → **(2) pollution percentage** → **(3) real-time clock**. The first line shows the title of the screen and the second line shows the value.

I²C communication: the LCD is driven through a PCF8574 I²C backpack [4] with address 0x27 (read with an I²C scanner sketch; a PCF8574A backpack would use 0x3F), and the AHT20 has address 0x38; both share one bus (SDA = GPIO21, SCL = GPIO22) at 100 kHz, the standard mode supported by the PCF8574. Because the addresses differ there is no conflict; the backpack jumpers must not be set to 0x38.

Voltage level: the LCD and its backpack are powered from 5 V, and their SDA/SCL pull-up resistors are usually tied to 5 V, while the ESP32 GPIO pins are only 3.3 V-tolerant.

In addition, a PCF8574 powered at 5 V expects a logic-high of about 3.5 V, slightly above what the ESP32 outputs. The team therefore uses a **bidirectional I²C level shifter** (BSS138-based) between the ESP32 (3.3 V side, shared with the AHT20) and the LCD backpack (5 V side), which solves both the protection and the noise-margin problem.

Non-blocking rotation: `millis()` is used instead of `delay()`. When `millis() - lastSwitch >= 3000`, the screen index is incremented and wraps back to the first screen after the third. Sensor reading, fan control and the MQTT connection are therefore never interrupted, whereas `delay(3000)` would block the whole program for 3 seconds. The display is cleared only when the screen changes, and values are refreshed once per second, to avoid flicker and unnecessary I²C traffic.

```
1 #include <LiquidCrystal_I2C.h>
2 LiquidCrystal_I2C lcd(0x27, 16, 2); // address, columns, rows
3
4 unsigned long lastSwitch = 0, lastRefresh = 0;
5 uint8_t screen = 0;
6
7 void updateLcd(float t, float pct, const char* timeStr) {
8     bool changed = false;
9     if (millis() - lastSwitch >= 3000) {
10         lastSwitch = millis();
11         screen = (screen + 1) % 3;
12         lcd.clear(); // clear only on screen change
13         changed = true;
14     }
15     if (changed || millis() - lastRefresh >= 1000) {
16         lastRefresh = millis();
17         lcd.setCursor(0, 0);
18         if (screen == 0) { lcd.print("Temperature");
19                             lcd.setCursor(0, 1); lcd.print(t, 1);  lcd.print(" C "); }
20         if (screen == 1) { lcd.print("Pollution");
21                             lcd.setCursor(0, 1); lcd.print(pct, 0); lcd.print(" % "); }
22         if (screen == 2) { lcd.print("Time");
23                             lcd.setCursor(0, 1); lcd.print(timeStr); }
24     }
25 }
```

Listing 2.2: Rotating the LCD screens with `millis()` (LiquidCrystal_I2C library).

Chapter 3

Firmware, MQTT Protocol and Control Logic (FSM)

The ESP32 is the edge node of the system. It reads the sensors, decides whether the fan should run, and exchanges data with the backend through MQTT. This chapter describes how the firmware communicates with the rest of the system (Sections 3.1 to 3.3), how the fan is controlled by a finite state machine that reduces chattering (Sections 3.4 and 3.5), and how the node keeps working when the network fails (Section 3.6). The firmware is written for the Arduino framework and uses the libraries `PubSubClient` (MQTT), `ArduinoJson` (payloads) and `Adafruit_AHTX0` (temperature and humidity sensor).

3.1 MQTT Communication Architecture

3.1.1 Topic Tree

All topics of the node follow the pattern `iot/kitchen/{device_id}/<topic>`, with `device_id` equal to `esp32_kitchen_01`. The fixed hierarchy allows the broker to apply access rules per device (Chapter 4) and lets a second kitchen be added later without changing the topic names. The firmware declares the seven topics as constants (Listing 3.1) and Figure 3.1 shows which of them the node publishes and which it subscribes to.

```
1 const char* DEVICE_ID = "esp32_kitchen_01";
2
3 const char* TOPIC_TELEMETRY      = "iot/kitchen/esp32_kitchen_01/telemetry";
4 const char* TOPIC_STATUS        = "iot/kitchen/esp32_kitchen_01/status";
5 const char* TOPIC_ACTUATOR_STATE = "iot/kitchen/esp32_kitchen_01/actuator/state";
6 const char* TOPIC_ACTUATOR_SET  = "iot/kitchen/esp32_kitchen_01/actuator/set";
7 const char* TOPIC_MODE_SET      = "iot/kitchen/esp32_kitchen_01/mode/set";
8 const char* TOPIC_CONFIG_STATE  = "iot/kitchen/esp32_kitchen_01/config/state";
```

```
9 const char* TOPIC_CONFIG_SET = "iot/kitchen/esp32_kitchen_01/config/set";
```

Listing 3.1: Topic constants of the firmware

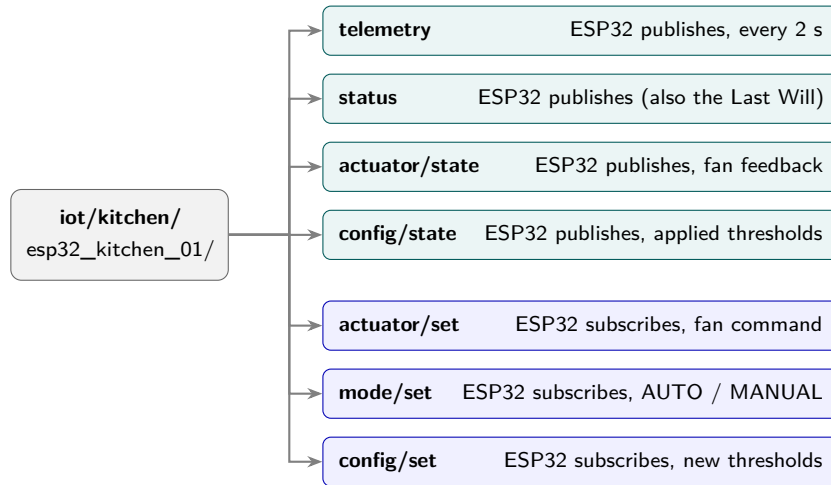


Figure 3.1: Topic tree of the node. Green: topics published by the ESP32. Blue: topics subscribed by the ESP32.

Table 3.1 summarises the meaning of each topic. The node only publishes data topics and only subscribes to command topics; the access control list of the broker enforces the same rule (Chapter 4).

Table 3.1: MQTT topics used by the ESP32 firmware

Topic	ESP32 action	Content and meaning
telemetry	Publish	Sensor values, fan state and mode, every 2 s
status	Publish and LWT	ONLINE after connecting, OFFLINE published by the broker if the link is lost
actuator/state	Publish	Fan state, duty and mode, with the reason of the last change
config/state	Publish	Thresholds currently applied by the node
actuator/set	Subscribe	Command to turn the fan on or off
mode/set	Subscribe	Switch between AUTO and MANUAL
config/set	Subscribe	New thresholds

3.1.2 Payload Structure

Every message is a JSON object built with `ArduinoJson`. The telemetry message is assembled by `sendTelemetry()` (Listing 3.2). It contains the identifier of the device, a sequence number `seq` that increases by one at every message, the sensor values, the state of the fan and the timestamp.

```

1 StaticJsonDocument<384> doc;
2 doc["device_id"] = DEVICE_ID;
3 doc["seq"] = msgSeq++;
4 doc["temperature"] = isnan(tempC) ? 0.0 : tempC;
5 doc["humidity"] = isnan(humi) ? 0.0 : humi;
6 doc["gas_ppm"] = ppm;
7 doc["fan_state"] = fanState;
8 doc["fan_duty"] = fanDuty;
9 doc["mode"] = currentMode;
10
11 time_t nowSec = time(nullptr);
12 uint64_t timestampMs = (uint64_t)nowSec * 1000ULL;
13 doc["timestamp"] = timestampMs;
14
15 char jsonBuffer[384];
16 serializeJson(doc, jsonBuffer);
17 bool ok = mqttClient.publish(TOPIC_TELEMETRY, jsonBuffer, false);

```

Listing 3.2: Building and publishing the telemetry message (`sendTelemetry`, excerpt)

Two fields deserve a comment. The `timestamp` is the UTC time in milliseconds since 1 January 1970. The ESP32 has no battery-backed clock, so it synchronises the time with an NTP server (pool.ntp.org) as soon as the Wi-Fi is connected. If the clock is not synchronised yet, the value is wrong, and the backend detects this and replaces it with its own arrival time (Chapter 4). The dashboard uses the same field to compute the data freshness (Chapter 5). The `seq` field lets the backend detect lost messages and lets the dashboard discard duplicated or old messages (Chapter 5).

The default packet buffer of `PubSubClient` is 256 bytes, and it must hold the topic name and the MQTT header in addition to the payload. A telemetry message together with its topic already uses most of this space, and configuration messages received from the backend can be longer. The firmware therefore enlarges the buffer with `mqttClient.setBufferSize(512)` before connecting; a message longer than the buffer is silently dropped and never sent.

Listing 3.3 shows the structure of the other messages, as produced or accepted by the firmware.

```

1 // status (retained; also registered as the Last Will with "OFFLINE")
2 {"device_id": "esp32_kitchen_01", "status": "ONLINE"}
3
4 // actuator/state (ESP32 -> Backend)
5 {"device_id": "esp32_kitchen_01", "fan_state": 1, "fan_duty": 255,
6  "mode": "MANUAL", "reason": "MANUAL_COMMAND"}
7
8 // config/state (ESP32 -> Backend, retained)
9 {"device_id": "esp32_kitchen_01", "pollution_threshold": 1000.0,

```

```

10 "temp_threshold": 33.0, "dwell_time_seconds": 10.0}
11
12 // actuator/set (Backend -> ESP32)
13 {"state": 1}
14
15 // mode/set (Backend -> ESP32)
16 {"mode": "MANUAL"}
17
18 // config/set (Backend -> ESP32); any subset of the fields is accepted
19 {"pollution_threshold": 1000, "temp_threshold": 38, "dwell_time_seconds": 30}

```

Listing 3.3: Structure of the other payloads (example values)

The reason field of actuator/state tells why the fan changed: `MANUAL_COMMAND` (command received by MQTT), `MODE_CHANGED` (mode switched from the dashboard), or `SERIAL_MANUAL_ON`, `SERIAL_MANUAL_OFF` and `SERIAL_AUTO` (commands typed on the serial console during tests).

3.1.3 Message Sequence

Figure 3.2 shows the messages exchanged between the four components in three situations: the normal telemetry flow, a manual fan command from the dashboard, and an unexpected loss of the node.

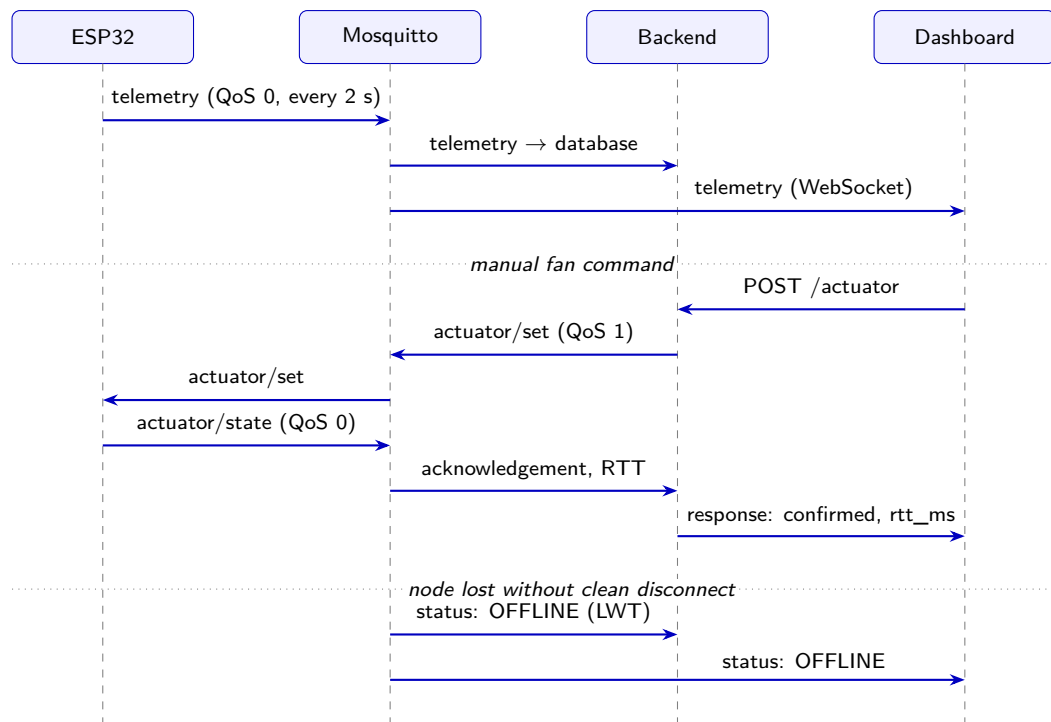


Figure 3.2: Sequence of the MQTT messages between the ESP32, the broker, the backend and the dashboard.

3.2 QoS Analysis and Selection

MQTT offers three levels of delivery guarantee. Each level adds reliability at the price of more messages, more memory on the sender and more delay (Table 3.2).

Table 3.2: MQTT quality of service levels

QoS	Guarantee	Handshake	Cost
0	At most once	None; the message is sent once	Lowest bandwidth and delay; a lost message is not recovered
1	At least once	PUBLISH, then PUBACK; resent if no PUBACK	One extra message; the receiver may see a duplicate
2	Exactly once	Four-step handshake	Highest delay and memory use; rarely justified on a small microcontroller

The levels actually used by the firmware are given in Table 3.3. The `PubSubClient` library can publish messages only with QoS 0, but it supports QoS 1 for the Last Will and for subscriptions. The firmware uses QoS 1 wherever the library allows it and where a lost message would matter.

Table 3.3: QoS and retain flag of each topic in the firmware

Topic	QoS	Retain	Reason
<code>telemetry</code>	0	no	Sent every 2 s; a lost sample is replaced by the next one
<code>status</code> (Last Will)	1	yes	The broker must deliver the offline state of the node
<code>status</code> (ONLINE message)	0	yes	A new client immediately sees the current state
<code>actuator/state</code>	0	no	Feedback after each change
<code>config/state</code>	0	yes	Lets the backend verify the applied thresholds at any time
<code>actuator/set, mode/set, config/set</code>	1 (subscription)	—	Commands must reach the node at least once

Telemetry with QoS 0. The node sends one sample every 2 seconds. If one sample

is lost, the next one arrives two seconds later and carries a newer value, so retransmitting the old one would only waste bandwidth and deliver outdated data. The `seq` field makes any loss visible without a protocol acknowledgement.

Commands with QoS 1. Commands are rare and important: a fan command that is lost means that the action of the user has no effect. The node subscribes to the three command topics with QoS 1 and the backend publishes them with QoS 1 (Chapter 4), so a command is delivered at least once. A duplicate is harmless because the commands are idempotent: setting the fan to “on” twice gives the same result as setting it once. QoS 2 is not needed for the same reason; its four-step handshake would add delay to the command round-trip time and use more memory on the ESP32 to protect against a problem that does not occur here.

Acknowledgement. The acknowledgement travels on `actuator/state` with QoS 0, so it can be lost. The backend does not wait for it forever: after its timeout it reports the command as not acknowledged (Chapter 4) and the user can send it again.

Retained messages. The retain flag is used where a client that connects later must know the current value immediately. The dashboard learns whether the node is online, and the backend reads the thresholds applied by the node. In the other direction, when the backend publishes `mode/set` and `config/set` as retained messages (Chapter 4), the node receives them again as soon as it subscribes after a reboot, so it recovers its mode and thresholds.

3.3 Last Will and Testament (LWT)

When a node is unplugged or loses its network, it cannot announce that it is leaving. The Last Will and Testament solves this: the node registers a message with the broker when it connects, and the broker publishes this message for the node if the connection ends without a proper disconnect.

Listing 3.4 shows the connection code of the firmware. The will is registered in the `connect()` call with the topic `status`, QoS 1, the retain flag set to `true` and the payload `{"device_id": "esp32_kitchen_01", "status": "OFFLINE"}`. Right after a successful connection the node publishes the `ONLINE` message on the same topic, also retained, and subscribes to the three command topics. Because both messages are retained, the topic `status` always holds the latest state of the node.

```
1 StaticJsonDocument<128> lwtDoc;  
2 lwtDoc["device_id"] = DEVICE_ID;  
3 lwtDoc["status"] = "OFFLINE";  
4 char lwtBuffer[128];  
5 serializeJson(lwtDoc, lwtBuffer);  
6
```

```

7 // connect(clientId, user, password, willTopic, willQoS, willRetain, willMessage)
8 if (mqttClient.connect(DEVICE_ID, MQTT_USER, MQTT_PASS,
9     TOPIC_STATUS, 1, true, lwtBuffer)) {
10     StaticJsonDocument<128> onlineDoc;
11     onlineDoc["device_id"] = DEVICE_ID;
12     onlineDoc["status"] = "ONLINE";
13     char onlineBuffer[128];
14     serializeJson(onlineDoc, onlineBuffer);
15     mqttClient.publish(TOPIC_STATUS, onlineBuffer, true);
16
17     mqttClient.subscribe(TOPIC_ACTUATOR_SET, 1);
18     mqttClient.subscribe(TOPIC_MODE_SET, 1);
19     mqttClient.subscribe(TOPIC_CONFIG_SET, 1);
20     sendConfigState();
21 }

```

Listing 3.4: Connection to the broker with the Last Will (`tryConnectMQTT`, excerpt)

Scenario. While the node is running normally, its power supply is cut. The broker stops receiving packets from the node. The firmware does not change the keep-alive interval of the library, which is 15 seconds by default, and the MQTT specification allows the broker to wait one and a half times this interval, about 22 seconds, before it considers the client lost. It then closes the connection and publishes the `OFFLINE` will message (Figure 3.3). The backend records a `DEVICE_OFFLINE` event and the dashboard changes its status. The sequence is the last part of Figure 3.2.

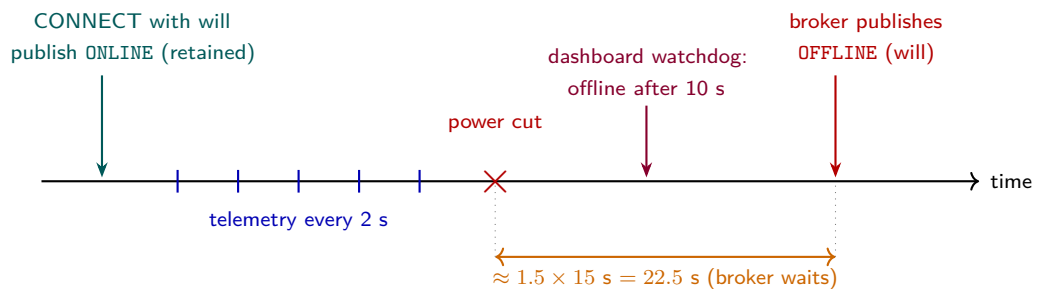


Figure 3.3: Timeline of an unexpected loss of the node: the will message is published after the keep-alive time.

The Last Will reacts only after the broker has noticed the broken connection, which takes longer than a few seconds. For this reason the dashboard does not depend on it alone: it also measures the age of the last valid telemetry packet and marks the node offline after 5 or 10 seconds (Chapter 5).

3.4 Finite State Machine for Fan Control

3.4.1 Inputs, Thresholds and Fan Levels

The automatic control is a finite state machine (FSM) with three states. It takes two inputs: the gas concentration P in ppm, computed from the MQ-135 after the calibration and the temperature and humidity compensation of Chapter 2, and the temperature T from the AHT20. The parameters are given in Table 3.4. The pollution threshold and the temperature threshold are variables, so they can be changed at run time with the topic `config/set`; the other values are constants of the program.

Table 3.4: Parameters of the fan control FSM

Parameter	Symbol	Value	Role
PPM_MID	P_{mid}	800 ppm [code currently has 1000]	Entering the MID state (constant)
pollutionThreshold	P_{bad}	1000 ppm	Entering the BAD state (variable)
PPM_HYST	H_P	50 ppm	Distance below P_{bad} needed to leave BAD
tempThreshold	T_{thr}	38 °C [code currently has 33.0]	Temperature limit for BAD (variable)
FAN_GOOD, FAN_MID, FAN_BAD	—	0, 150, 255	PWM duty (8 bit) of each state

The three states and the fan level of each state are:

GOOD: the air is clean, the fan is off (duty 0).

MID: moderate pollution, the fan runs at a duty of 150 out of 255, about 60%.

BAD: high pollution or high temperature, the fan runs at full speed (duty 255).

The fan is driven through a MOSFET on GPIO 26 with PWM at 5000 Hz and 8-bit resolution, that is 256 power levels. A frequency of 20 to 25 kHz would move the switching noise out of the audible range, but it increases the switching losses and the heating of the MOSFET. We selected 5 kHz as a compromise between smooth speed control, acceptable noise and a cool MOSFET. The constants and the function that writes the duty are shown in Listing 3.5. The preprocessor test on `ESP_ARDUINO_VERSION_MAJOR` makes the same source compile with version 2 and version 3 of the ESP32 Arduino core, which use different LEDC functions.

```

1 #define FAN_FREQ    5000    // Hz
2 #define FAN_RES     8       // 8 bit: duty 0..255
3 #define FAN_GOOD    0       // fan off
4 #define FAN_MID     150     // ~60% speed
5 #define FAN_BAD     255     // 100% speed
6
7 void fanWrite(uint8_t duty) {
8     fanDuty = duty;
9     fanState = (duty > 0) ? 1 : 0;
10 #if ESP_ARDUINO_VERSION_MAJOR >= 3
11     ledcWrite(PIN_FAN, duty);
12 #else
13     ledcWrite(FAN_CH, duty);
14 #endif
15 }

```

Listing 3.5: PWM constants and fan output function

3.4.2 State Logic

The state is updated by `updateStatus()` (Listing 3.6), which runs once per second after the sensors are read. A boolean variable `alarmOn` remembers whether the BAD state is active. It is set when the gas concentration exceeds P_{bad} or the temperature exceeds T_{thr} , and it is cleared only when the concentration has fallen below $P_{bad} - H_P$ and the temperature is back under its limit. When the alarm is off, the state is MID if $P \geq P_{mid}$ and GOOD otherwise. The fan is written only in AUTO mode.

```

1 void updateStatus() {
2     bool wasOn = alarmOn;
3
4     if (ppm > pollutionThreshold || (!isnan(tempC) && tempC > tempThreshold)) {
5         alarmOn = true;
6     } else if (ppm < pollutionThreshold - PPM_HYST && tempC <= tempThreshold) {
7         alarmOn = false;
8     }
9
10    if (alarmOn)          airStatus = AIR_BAD;
11    else if (ppm >= PPM_MID) airStatus = AIR_MID;
12    else                  airStatus = AIR_GOOD;
13
14    if (currentMode == "AUTO") {
15        fanWrite(airStatus == AIR_BAD ? FAN_BAD
16                : airStatus == AIR_MID ? FAN_MID : FAN_GOOD);
17    }
18
19    if (alarmOn != wasOn) {

```

```

20     Serial.println(alarmOn ? "[ALARM] ON - threshold exceeded!"
21                       : "[ALARM] OFF - back to safe level");
22 }
23 }

```

Listing 3.6: State update and fan control (`updateStatus`)

3.4.3 State Diagram

Figure 3.4 shows the states and the conditions of every transition. Leaving the BAD state is stricter than entering it, because of the hysteresis H_P .

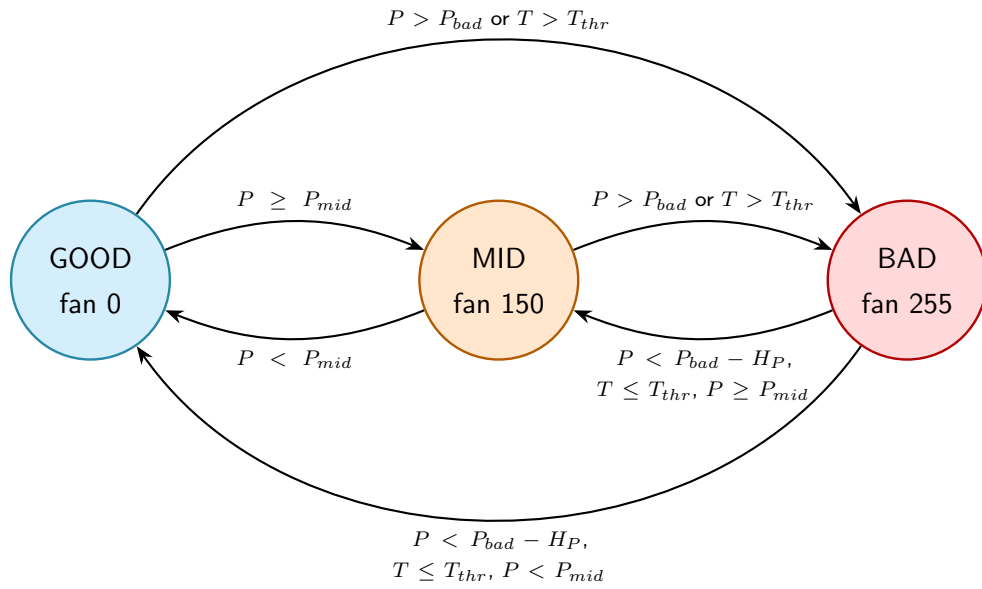


Figure 3.4: State diagram of the fan control FSM.

3.4.4 Truth Table

Table 3.5 gives the next state for every combination of the previous alarm state and the inputs, with $P_{mid} = 800$, $P_{bad} = 1000$ and $H_P = 50$.

Table 3.5: Next state of the FSM

Previous alarmOn	Condition	Next state
false	$P > 1000$ or $T > T_{thr}$	BAD
false	$800 \leq P \leq 1000$ and $T \leq T_{thr}$	MID
false	$P < 800$ and $T \leq T_{thr}$	GOOD
true	$P > 1000$ or $T > T_{thr}$	BAD
true	$950 \leq P \leq 1000$ and $T \leq T_{thr}$ (hysteresis zone)	BAD (kept)
true	$800 \leq P < 950$ and $T \leq T_{thr}$	MID
true	$P < 800$ and $T \leq T_{thr}$	GOOD

3.4.5 Why Two Variables

A single gas threshold would ignore a second danger of cooking: heat. A pan left on a strong flame can raise the temperature of the small kitchen before the gas sensor reacts, and the MQ-135 itself is sensitive to temperature (Chapter 2). The FSM therefore enters the BAD state when either the gas concentration or the temperature is above its limit, so the fan is started by whichever danger appears first. Leaving BAD is stricter: both variables must be back in the safe zone, so the fan does not stop while one of them is still high.

3.4.6 AUTO and MANUAL Modes

In AUTO mode the FSM decides. A fan command from MQTT or from the serial console switches the node to MANUAL mode (Listing 3.7): the fan then follows the commands only. `updateStatus()` keeps running in MANUAL mode, because it only writes the fan when the mode is AUTO, so the node continues to compute the state and to print the [ALARM] message when a limit is exceeded. A fan that was switched off by mistake therefore does not hide a real hazard.

```

1 if (topicStr == TOPIC_ACTUATOR_SET) {
2     int newState = -1;
3     if (doc.containsKey("state")) {
4         newState = doc["state"].is<bool>() ? (doc["state"].as<bool>() ? 1 : 0)
5                                     : doc["state"].as<int>();
6     }
7     if (newState != -1) {
8         currentMode = "MANUAL";
9         fanWrite(newState ? FAN_BAD : FAN_GOOD);
10        sendActuatorState("MANUAL_COMMAND");
11    }
12 }

```

```

13 else if (topicStr == TOPIC_MODE_SET) {
14     if (doc.containsKey("mode")) {
15         currentMode = doc["mode"].as<String>();
16         updateStatus();
17         sendActuatorState("MODE_CHANGED");
18     }
19 }
20 else if (topicStr == TOPIC_CONFIG_SET) {
21     if (doc.containsKey("pollution_threshold")) pollutionThreshold = doc["
    ↪ pollution_threshold"];
22     if (doc.containsKey("temp_threshold"))      tempThreshold      = doc["
    ↪ temp_threshold"];
23     if (doc.containsKey("dwell_time_seconds")) dwellTimeSeconds = doc["
    ↪ dwell_time_seconds"];
24     updateStatus();
25     sendConfigState();
26 }

```

Listing 3.7: Handling of the command topics (`mqttCallback`, excerpt)

After each change, `sendActuatorState()` publishes the feedback message of Listing 3.3 on `actuator/state`, which the backend uses to confirm the command and measure the round-trip time. After a change of thresholds, `sendConfigState()` publishes the applied values on `config/state`, so the backend can check that the node uses the thresholds it sent.

3.5 Chattering Prevention

3.5.1 The Problem

Chattering happens when a sensor value fluctuates around a threshold and the controller switches the output on and off again and again. In our system the MQ-135 reading always contains noise, and electromagnetic interference from the fan motor adds to it. With one fixed threshold, the fan would start and stop many times per minute. This shortens the life of the MOSFET, which heats up at every switching, and the back electromotive force of the motor coil produces voltage spikes on the board. For the user, a fan that jerks on and off is also an annoyance.

3.5.2 Hysteresis

The threshold for entering the BAD state and the threshold for leaving it are different. The fan goes to full speed when $P > P_{bad}$ but returns to a lower level only when $P < P_{bad} - H_P$, with $H_P = 50$ ppm (PPM_HYST). Noise smaller than this band cannot toggle the fan. In

Listing 3.6 this is the difference between the condition that sets `alarmOn` and the condition that clears it, and the variable `alarmOn` keeps the state in the zone between the two thresholds. Figure 3.5 illustrates the principle: the concentration dips below 1000 ppm twice while the fan is at full speed, but it never falls below 950 ppm, so the fan keeps running.

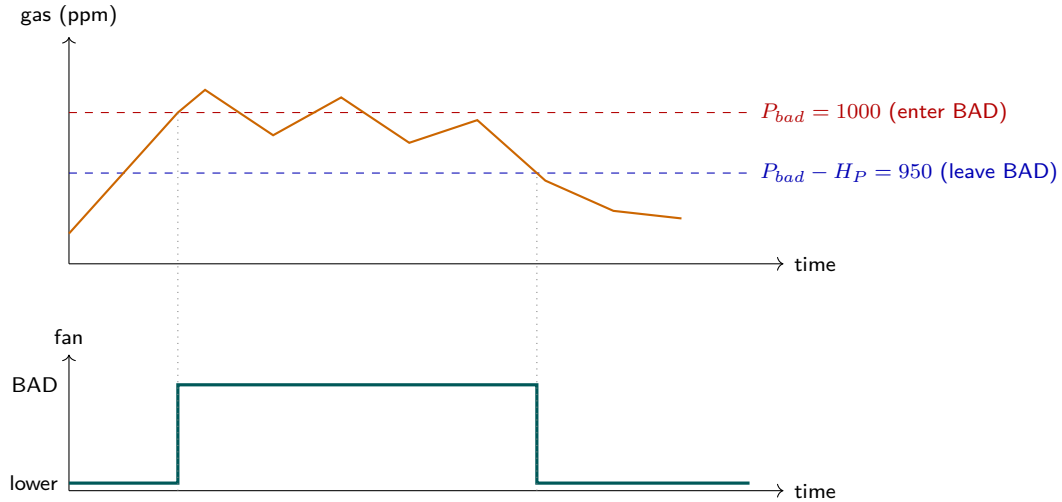


Figure 3.5: Principle of the hysteresis band (schematic). Fluctuations between 950 and 1000 ppm do not change the state.

3.5.3 Limits of the Current Implementation and Dwell Time

The hysteresis protects only the boundary of the BAD state. The boundary between GOOD and MID uses a single threshold (P_{mid}), so a concentration that fluctuates around it can still toggle the fan between 0 and 150. The data contract also contains a parameter `dwell_time_seconds`: the node stores it, accepts it on `config/set` and reports it on `config/state`, but the present version of the firmware does not yet use it to lock the state. A minimum dwell time would hold each new state for a fixed time (for example 30 s) before another change is allowed, and would remove the remaining short toggles. It is the natural next improvement of the controller.

3.6 Network Loss Handling

3.6.1 Non-Blocking Program Structure

A call to `delay()` inside `loop()` stops the CPU. During that time the MQTT and Wi-Fi libraries cannot serve their keep-alive packets, incoming messages overflow the buffer, and the connection may be dropped. In the firmware, `delay()` appears only in `setup()` (start-up self-test, Wi-Fi wait, sensor warm-up) and in the R0 calibration started by the

operator. The main `loop()` never blocks: each task compares the current time from `millis()` with the time of its last run and executes only when its period has elapsed (Listing 3.8, Table 3.6).

```
1 uint32_t now = millis();
2
3 // 1. Read the sensors and update the logic every 1 second
4 if (now - lastSensor >= SENSOR_MS) {
5     lastSensor = now;
6     readAHT();
7     readAir();
8     updateStatus();
9     drawPage();
10    printSerial();
11 }
12
13 // 2. Switch the LCD page every 2 seconds
14 if (now - lastPage >= PAGE_MS) {
15     lastPage = now;
16     page ^= 1;
17     drawPage();
18 }
19
20 // 3. Send the telemetry every 2 seconds
21 if (now - lastTelemetryTime >= 2000) {
22     lastTelemetryTime = now;
23     sendTelemetry();
24 }
25
26 // 4. Handle the serial commands
27 handleSerial();
```

Listing 3.8: Non-blocking scheduling in `loop()` (excerpt)

Table 3.6: Periodic tasks of the firmware

Task	Period	Content
Sensors and logic	1000 ms	Read the AHT20 and the MQ-135, compensate, update the state and the fan
Display	2000 ms	Alternate the two pages of the 16×2 LCD (temperature and humidity; gas level, mode and state)
Telemetry	2000 ms	Build the JSON and publish it on telemetry
MQTT connection	every loop; retry every 5 s	<code>mqttClient.loop()</code> while connected, reconnection otherwise
Wi-Fi connection	every loop; retry every 10 s	Check the status and restart the connection
Serial commands	every loop	Calibration, manual fan, AUTO mode

The counter returned by `millis()` is an unsigned 32-bit integer and returns to 0 after about 49.7 days. The test `now - lastSensor >= SENSOR_MS` is a subtraction of unsigned numbers, which gives the correct elapsed time even when the counter has overflowed between the two readings.

3.6.2 Online Mode and Local Autonomous Fallback

The node has two operating modes (Figure 3.6). In the online mode it works normally and exchanges data with the broker. When the Wi-Fi or the MQTT connection is lost, it switches to the fallback mode. In this mode the telemetry cannot be delivered: `sendTelemetry()` prints a warning on the serial console and skips the message. The sensor task, the state machine and the fan output keep running without interruption, because they do not depend on the network. The node then behaves as an independent safety device: a gas leak is still detected and the fan is still started even if the router or the cloud is down. When the connection returns, the node goes back to the online mode, publishes **ONLINE** again, subscribes to the command topics and publishes its thresholds.

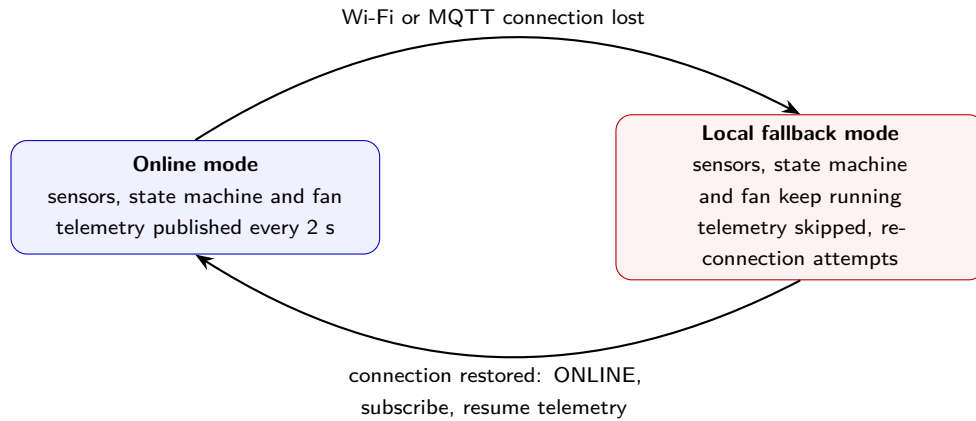


Figure 3.6: Transition between the online mode and the local autonomous fallback mode.

3.6.3 Reconnection Strategy

The reconnection is also non-blocking (Listing 3.9). The firmware stores the time of the last attempt and tries again only when the retry period has elapsed, so a failed connection never freezes the sensor task or the state machine. If the Wi-Fi is lost, a new connection is started every 10 seconds; when the Wi-Fi is available, the connection to the broker is retried every 5 seconds until it succeeds. The NTP configuration is repeated after a Wi-Fi reconnection if it was not done at start-up, so the timestamps become valid as soon as the network returns.

```

1 // in loop()
2 if (WiFi.status() != WL_CONNECTED) {
3     uint32_t nowW = millis();
4     if (nowW - lastWifiRetryTime > 10000) {           // retry Wi-Fi every 10 s
5         lastWifiRetryTime = nowW;
6         WiFi.disconnect();
7         WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
8     }
9 } else {
10     tryConnectMQTT();
11     if (mqttClient.connected()) mqttClient.loop();
12 }
13
14 // in tryConnectMQTT()
15 unsigned long now = millis();
16 if (now - lastMqttRetryTime > 5000) {               // retry the broker every 5 s
17     lastMqttRetryTime = now;
18     // ... connect with the Last Will (Listing 3.5)
19 }
  
```

Listing 3.9: Non-blocking reconnection (loop, excerpt, and the retry timer of tryConnectMQTT)

Possible improvement: exponential backoff. A fixed retry period is simple, but

it keeps a fixed load on the broker during a long outage. With an exponential backoff, the waiting time doubles after each failure up to a limit and returns to the initial value after a success (Table 3.7). The node then retries quickly after a short glitch and slowly during a long outage, which reduces the traffic on the broker and the use of the Wi-Fi radio. Only the two constants 5000 and 10000 of Listing 3.9 would have to be replaced by a variable that is doubled after each failed attempt.

Table 3.7: Waiting time of the proposed exponential backoff

Attempt	Wait before the next attempt
1	2 s
2	4 s
3	8 s
4	16 s
5	32 s
6 and later	60 s (maximum)

Chapter 4

Backend, Database and Alerting System

4.1 Overview of Responsibilities and Backend Architecture

In the end-to-end architecture of the smart kitchen environment monitoring and automatic ventilation system, the backend is the middleware layer between the ESP32 sensor node and the Web Dashboard. As the owner of this layer, Member 3 is responsible for:

1. operating the Eclipse Mosquitto MQTT broker with password authentication and topic-level access control;
2. ingesting the telemetry stream (one message every 2 seconds), validating it against the team's data contract (version 1.2) and storing it in TimescaleDB;
3. exposing a REST API built with FastAPI for monitoring, manual control and remote configuration;
4. forwarding control commands to the ESP32 and confirming every command through the device acknowledgement, which also yields the command round-trip time;
5. dispatching alerts to a Discord channel through a webhook;
6. serving the LSTM risk-forecasting model developed by Member 5 and storing every forecast;
7. deploying the whole stack on an AWS EC2 virtual machine with Docker Compose and HTTPS.

Every server component runs in its own Docker container on one AWS EC2 virtual machine. Figure 4.1 summarises the data flows: the ESP32 communicates only with the broker over MQTT port 1883, the backend is the only component allowed to issue commands, and the browser reaches the system only through Caddy. Live values arrive in the browser over MQTT-over-WebSocket, while history, control and configuration go through the REST API.

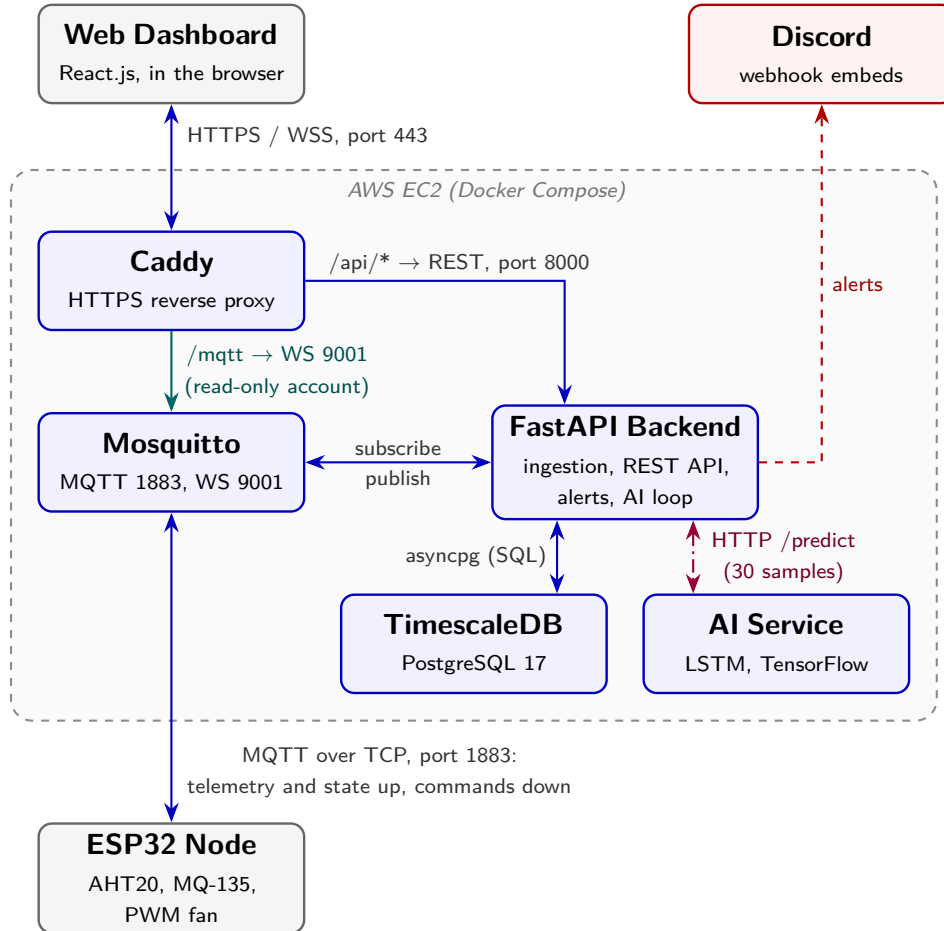


Figure 4.1: Backend architecture and data flows. Every block inside the dashed box is a Docker container on the AWS EC2 server; the browser and the ESP32 reach it only through ports 443 and 1883.

4.2 Mosquitto MQTT Broker and Access Control

The system uses Eclipse Mosquitto 2 (official Docker image `eclipse-mosquitto:2`) as the central MQTT broker. It exposes two listeners:

Port 1883 (MQTT over TCP): used by the ESP32 and by the backend. It is the only broker port open to the Internet.

Port 9001 (MQTT over WebSockets): used by the Web Dashboard, because a browser cannot open a raw TCP socket. On the server this port is bound to

127.0.0.1; browsers reach it only through the Caddy reverse proxy at `wss://iot-kitchen-hcmute.duckdns.org/mqtt`.

Anonymous access is disabled. The password file and the access control list (ACL) are not stored in the repository: they are generated inside the container at every start-up from the passwords in `.env` and from the template `acl.template`, in which the placeholder `__DEVICE_ID__` is replaced by the configured device identifier. The generated files belong to the `mosquitto` user with permission `0600`, and the container health check subscribes to the broker with real credentials, so a healthy status proves that both networking and authentication work.

```
1 # Port 1883: MQTT over TCP for the ESP32 and the backend
2 listener 1883
3 protocol mqtt
4
5 # Port 9001: MQTT over WebSockets for the browser dashboard
6 listener 9001
7 protocol websockets
8
9 # No anonymous clients. Both files are generated in the container at start-up
10 # from .env, so the passwords never enter the Git repository.
11 allow_anonymous false
12 password_file /mosquitto/runtime/passwd
13 acl_file /mosquitto/runtime/acl
14
15 # Keep retained messages (status, mode/set, config/set) across broker restarts
16 persistence true
17 persistence_location /mosquitto/data/
18
19 # Log to stdout: docker compose logs -f mosquitto
20 log_dest stdout
21 log_type error
22 log_type warning
23 log_type notice
24 log_type information
25 connection_messages true
26 log_timestamp true
```

Listing 4.1: Mosquitto broker configuration (`mosquitto/config/mosquitto.conf`, comments translated)

The ACL follows the principle of least privilege (Table 4.1). The dashboard account is read-only because its password is embedded in the browser JavaScript, and the ESP32 account may publish only the data topics of its own device, so a compromised node can neither forge fan commands nor overwrite the data of another kitchen.

Table 4.1: MQTT accounts and their ACL permissions

Account	Used by	Permissions
backend_svc	FastAPI backend	Read and write <code>iot/kitchen/#</code> ; read <code>\$/SYS/#</code>
esp32_device	ESP32 node	Only the topics of its own device: read and write <code>telemetry</code> ; write <code>status</code> , <code>actuator/state</code> , <code>config/state</code> ; read <code>actuator/set</code> , <code>mode/set</code> , <code>config/set</code>
web_dashboard	Browser dashboard	Read <code>iot/kitchen/#</code> only

All topics follow the pattern `iot/kitchen/{device_id}/<topic>`. Table 4.2 lists the topics defined by the data contract. Retained messages are used wherever a newly connected client must immediately know the current value: the dashboard learns whether the device is online, and the ESP32 recovers its operating mode and thresholds after a reboot. Broker persistence keeps these retained messages across broker restarts.

Table 4.2: MQTT topics defined by the data contract v1.2

Topic	Direction	QoS	Retain	When it is sent
<code>telemetry</code>	ESP32 → Backend	0	no	Every 2 seconds
<code>status</code>	ESP32 → Backend (LWT)	1	yes	On connection; OFFLINE is published by the broker when the link is lost
<code>actuator/state</code>	ESP32 → Backend	1	yes	When the fan changes or a command is answered
<code>config/state</code>	ESP32 → Backend	1	yes	At start-up and after every <code>config/set</code>
<code>actuator/set</code>	Backend → ESP32	1	no	When the user switches the fan
<code>mode/set</code>	Backend → ESP32	1	yes	When the operating mode changes
<code>config/set</code>	Backend → ESP32	1	yes	When a threshold changes
<code>ai_prediction</code>	Backend → Dashboard	0	no	Every 10 seconds while fresh telemetry arrives

The `status` topic carries the Last Will and Testament (LWT). When the ESP32 connects it publishes `{"device_id": "esp32_kitchen_01", "status": "ONLINE"}` with the retain flag and registers an **OFFLINE** will message, which the broker publishes automatically if the connection is lost without a clean disconnect.

4.3 Time-Series Database Schema Design

IoT telemetry has distinct time-series characteristics: a steady write rate (one message every 2 seconds), append-only insertion, and queries that almost always select a time range, either for charts or for the most recent window used by the forecasting model.

We select **PostgreSQL 17 with the TimescaleDB extension** (image `timescale/timescaledb:latest-pg17`). It keeps standard SQL and relational tables, while *hypertables* transparently partition time-series tables into chunks by time. The telemetry hypertable uses 1-day chunks, so recent data stays small and hot, and a retention policy removes data older than 90 days by dropping whole chunks instead of deleting rows one by one. The `time_bucket()` function aggregates samples into fixed intervals for the history charts; the forecasting model reads the raw 2-second samples directly.

4.3.1 Database Schema Specification

The database contains five tables (Table 4.3); the telemetry table is detailed in Table 4.4.

Table 4.3: Tables of the `iot_kitchen` database

Table	Type	Content
<code>telemetry</code>	Hypertable, 1-day chunks, 90-day retention	One row per telemetry message
<code>system_events</code>	Regular table	Event log: device online/offline, commands, fan changes, configuration synchronisation, sensor faults, alerts, authentication failures
<code>device_config</code>	Regular table, one row per device	Desired thresholds: FSM parameters (<code>ppm_mid_threshold</code> , <code>ppm_bad_threshold</code> , <code>temp_threshold</code> , <code>dwel_time_seconds</code>) and alert thresholds
<code>ai_forecasts</code>	Hypertable, 7-day chunks, 90-day retention	Every LSTM forecast: risk class 15 minutes ahead, confidence, class probabilities and target time
<code>ai_predictions</code>	Hypertable, 7-day chunks, 90-day retention	Forecasts of the former regression model, kept for reference

The sensor columns are nullable on purpose. An out-of-range reading, such as `-999` from a disconnected sensor, is stored as `NULL` and logged as a `SENSOR_FAULT` event, so one faulty sensor does not discard the valid readings of the others. The composite index on `(device_id, time DESC)` serves both the latest-value query and the time-range queries of the charts.

Table 4.4: Telemetry table schema (`telemetry`)

Column Name	Data Type	Constraint	Description
<code>time</code>	TIMESTAMPTZ	NOT NULL	Sampling time sent by the ESP32 (UTC, milliseconds)
<code>received_at</code>	TIMESTAMPTZ	NOT NULL	Arrival time at the backend (default <code>now()</code>); <code>received_at</code> – <code>time</code> is the end-to-end latency
<code>device_id</code>	TEXT	NOT NULL	Device identifier, e.g. <code>esp32_kitchen_01</code>
<code>seq</code>	BIGINT		Message counter; gaps reveal lost messages
<code>temperature</code>	DOUBLE PRECISION		Ambient temperature (°C)
<code>humidity</code>	DOUBLE PRECISION		Relative humidity (%RH)
<code>gas_ppm</code>	DOUBLE PRECISION	≥ 0	Gas concentration from the MQ-135 calibration curve; drives the FSM and the alerts
<code>pollution_percent</code>	DOUBLE PRECISION	0–100	Pollution index used as the AI model input
<code>rs_ro_ratio</code>	DOUBLE PRECISION		MQ-135 resistance ratio R_s/R_0
<code>fan_state</code>	SMALLINT	0 or 1	Fan off / on
<code>fan_speed_percent</code>	SMALLINT	0–100	Actual fan speed: 0, 50 or 100
<code>mode</code>	TEXT	AUTO or MANUAL	Operating mode
<code>network_status</code>	TEXT		Connection status reported by the node

```

1 CREATE EXTENSION IF NOT EXISTS timescaledb;
2
3 CREATE TABLE IF NOT EXISTS telemetry (
4     time                TIMESTAMPTZ NOT NULL,           -- sampling time (ESP32)
5     received_at         TIMESTAMPTZ NOT NULL DEFAULT now(), -- arrival time (latency)
6     device_id           TEXT NOT NULL,
7     seq                 BIGINT,                         -- message counter
8     gas_ppm             DOUBLE PRECISION CHECK (gas_ppm >= 0),
9     fan_speed_percent   SMALLINT CHECK (fan_speed_percent BETWEEN 0 AND 100),
10    temperature         DOUBLE PRECISION,               -- degrees Celsius
11    humidity            DOUBLE PRECISION,               -- %RH
12    pollution_percent   DOUBLE PRECISION CHECK (pollution_percent BETWEEN 0 AND 100),
13    rs_ro_ratio         DOUBLE PRECISION,               -- MQ-135 Rs/RO
14    fan_state           SMALLINT CHECK (fan_state IN (0, 1)),
15    mode                TEXT CHECK (mode IN ('AUTO', 'MANUAL')),
16    network_status      TEXT
17 );
18
19 -- Hypertable with 1-day chunks; raw data is kept for 90 days
20 SELECT create_hypertable('telemetry', by_range('time', INTERVAL '1 day'),
21                          if_not_exists => TRUE);
22 CREATE INDEX IF NOT EXISTS idx_telemetry_device_time
23     ON telemetry (device_id, time DESC);
24 SELECT add_retention_policy('telemetry', INTERVAL '90 days', if_not_exists => TRUE);

```

Listing 4.2: Telemetry hypertable definition (database/init/10_schema.sql, reformatted, comments translated)

4.4 Backend Server Architecture and Ingestion Service

The backend is written in Python 3.12 with the asynchronous **FastAPI** framework. It uses **asyncpg** for database access and **aiomqtt** for a single persistent MQTT connection authenticated as **backend_svc**. Besides serving HTTP requests, the process runs three background tasks: the MQTT service, the alert delivery worker and the AI prediction loop.

The MQTT service subscribes to four topic filters and handles each message type as follows:

1. **Telemetry** (iot/kitchen/+/telemetry, QoS 0): out-of-range readings are converted to null, the payload is validated with Pydantic against the data contract, the sample time is checked, sensor faults and sequence gaps are recorded, the row is inserted into TimescaleDB and the alert rules are evaluated (Listing 4.3).
2. **Status and LWT** (iot/kitchen/+/status, QoS 1): ONLINE and OFFLINE transitions are stored as **DEVICE_ONLINE** and **DEVICE_OFFLINE** events. The retained status received when the backend itself reconnects only updates the in-memory state,

so events are not duplicated.

3. **Actuator state** (iot/kitchen/+/actuator/state, QoS 1): completes pending fan commands (acknowledgement and round-trip time) and records fan changes. The reason field distinguishes FSM, MANUAL_COMMAND, SAFETY_OVERRIDE and IGNORED_AUTO_MODE.
4. **Configuration state** (iot/kitchen/+/config/state, QoS 1): the thresholds reported by the device are compared with device_config. The result is logged as CONFIG_CONFIRMED or CONFIG_MISMATCH and exposed as config_in_sync in /status.

Timestamp policy. The sample time is normally the timestamp sent by the ESP32, because it reflects when the measurement was taken. The backend falls back to its own receive time when the device clock is more than 60 seconds in the future or more than 24 hours in the past, which typically happens before NTP has synchronised, and records a CLOCK_SKEW event (at most once per minute). A lag of more than five minutes is still stored with the device time, because it may be buffered data, but it is also reported as CLOCK_SKEW.

Failure visibility. A failing database write, for example after a schema change without the corresponding migration, is reported once as a DB_WRITE_FAILED event and switches /health to degraded; DB_WRITE_RECOVERED is logged when writes succeed again. Without this, the dashboard, which reads live values directly from MQTT, would keep looking healthy while no data was being stored.

```
1 async def _on_telemetry(self, device_id: str, data: dict[str, Any]) -> None:
2     # Turn non-physical readings (-1, -999, NaN...) into null BEFORE validation,
3     # so that one broken sensor does not discard the whole message
4     cleaned, invalid = sanitize_readings(data)
5     t = TelemetryIn.model_validate(cleaned)
6
7     received_at = datetime.now(timezone.utc)
8     sample_time = await self._sample_time(device_id, t.timestamp, received_at)
9
10    await self._check_sensor_faults(device_id, invalid, data)
11    self._check_seq(device_id, t.seq)
12
13    try:
14        await self.db.insert_telemetry(device_id, sample_time, received_at, t)
15    except Exception as exc:
16        error = f"{type(exc).__name__}: {exc}"
17        if error != self.last_insert_error: # log once per new error
18            await self.db.log_event(device_id, "DB_WRITE_FAILED",
19                                   f"Cannot write telemetry: {error}",
```

```

20         "CRITICAL", "backend")
21     self.last_insert_error = error
22 else:
23     if self.last_insert_error:
24         await self.db.log_event(device_id, "DB_WRITE_RECOVERED",
25                                 "Telemetry writes work again",
26                                 "INFO", "backend")
27         self.last_insert_error = None
28
29     if t.mode:
30         self.latest_mode[device_id] = t.mode
31     await self.alerts.evaluate(device_id, t, sample_time)

```

Listing 4.3: Telemetry ingestion pipeline (backend/app/mqtt_service.py, excerpt, reformatted, comments and messages translated)

4.5 REST API Specification

All endpoints are served under the prefix `/api/v1/kitchen`; the public base URL is `https://iot-kitchen-hcmute.duckdns.org/api/v1/kitchen` through Caddy. FastAPI generates the interactive documentation at `/docs` and the OpenAPI specification at `/openapi.json`. An optional `device_id` query parameter selects the device; when it is omitted, the `DEVICE_ID` configured in `.env` is used.

Table 4.5: Backend REST API endpoints (prefix `/api/v1/kitchen`; * requires the `X-API-Key` header)

Method	Endpoint	Parameters	Description & Response
GET	<code>/status</code>	–	Latest telemetry, device status, data freshness (FRESH < 3 s, DELAYED ≤ 10 s, STALE), active alerts, thresholds applied by the device and <code>config_in_sync</code>
GET	<code>/history</code>	<code>start</code> , <code>end</code> , <code>bucket_seconds</code> , <code>order</code> , <code>limit</code> , <code>offset</code>	Raw samples or time-bucket averages for charts
GET	<code>/events</code>	<code>limit</code> , <code>event_type</code>	System event log, newest first
GET	<code>/diagnostics</code>	<code>minutes</code> , <code>expected_interval_s</code>	Packet loss from <code>seq</code> , latency (average, p95, maximum) and data-contract checks
GET	<code>/config</code>	–	Thresholds stored in <code>device_config</code>
POST*	<code>/config</code>	JSON with the thresholds to change	Validates and stores the values, then publishes all four FSM thresholds on <code>config/set</code> (retained)
POST*	<code>/actuator</code>	<code>{"state": 0 1}</code>	Publishes <code>actuator/set</code> and waits up to 3 s for the ESP32 acknowledgement; returns <code>confirmed</code> and <code>rtt_ms</code>
POST*	<code>/mode</code>	<code>{"mode": "AUTO" "MANUAL"}</code>	Publishes <code>mode/set</code> (retained)
GET	<code>/prediction</code>	–	Latest LSTM forecast and AI service status; <code>available</code> is true only while forecasts are fresh, otherwise <code>detail</code> gives the reason
GET	<code>/prediction/history</code>	<code>minutes</code> , <code>limit</code>	Stored LSTM forecasts, newest first
GET	<code>/prediction/accuracy</code>	<code>hours</code>	MAE and RMSE of the former regression model (legacy)
POST*	<code>/alerts/test</code>	–	Sends a test alert to Discord through the configured webhook

In addition, `GET /health` (outside the prefix) reports the state of the database, the broker connection and the Discord notification channel; it is used by the Docker health check.

Access protection. When `API_KEY` is set in `.env`, every write endpoint requires

the header `X-API-Key`, which is compared in constant time. After ten failed attempts within one minute from the same IP address the API answers HTTP 429 and stops logging further `AUTH_FAILED` events, so key guessing cannot flood the event table. Setting `READ_ONLY=true` blocks every write endpoint, which is useful for public demonstrations, and the allowed CORS origins are configured through `CORS_ORIGINS`.

Command path. `POST /actuator` does not stop at publishing. The MQTT service registers a waiter before it publishes `{"command": "SET_FAN", "state": 0|1}` with QoS 1, and completes the waiter when the ESP32 answers on `actuator/state` (Listing 4.4). The measured `rtt_ms` is returned to the dashboard and stored with the `MANUAL_FAN_COMMAND` event, which provides the data for the control-latency test. The ESP32 executes manual commands only in `MANUAL` mode; in `AUTO` mode it answers with the reason `IGNORED_AUTO_MODE`, and the API returns a warning asking the dashboard to switch to `MANUAL` first.

```
1 async def send_fan_command(self, device_id: str, state: int) -> dict[str, Any]:
2     topic_name = topic(device_id, "actuator/set")
3     payload = {"command": "SET_FAN", "state": state}
4
5     future: asyncio.Future = asyncio.get_running_loop().create_future()
6     self._ack_waiters.setdefault(device_id, []).append(future) # register first
7     started = time.perf_counter()
8     try:
9         await self.publish_json(topic_name, payload, qos=1, retain=False)
10        ack: ActuatorStateIn = await asyncio.wait_for(
11            future, timeout=self.s.command_ack_timeout_s)
12        return {"acknowledged": True, "fan_state": ack.value, "reason": ack.reason,
13              "rtt_ms": round((time.perf_counter() - started) * 1000, 1),
14              "topic": topic_name, "payload": payload}
15    except TimeoutError:
16        return {"acknowledged": False, "fan_state": None, "reason": None,
17              "rtt_ms": None, "topic": topic_name, "payload": payload}
18    finally:
19        waiters = self._ack_waiters.get(device_id, [])
20        if future in waiters:
21            waiters.remove(future)
```

Listing **4.4:**

Fan command with acknowledgement and round-trip time (backend/app/mqtt_service.py, reformatted)

4.6 Discord Webhook Alerting System Integration

The backend evaluates two alert rules on every telemetry message (Table 4.6). Their thresholds are stored in `device_config` and can be changed at run time through `POST`

/config.

Table 4.6: Alert rules

Rule	Condition	Default threshold	Hysteresis
POLLUTION_HIGH	gas_ppm > alert_ppm_threshold	1000 ppm	50 ppm
TEMPERATURE_HIGH	temperature > alert_temp_threshold	40 °C	2 °C

Each rule works as an **episode state machine** (Listing 4.5). When a new episode starts, an ALERT message is sent, but only if at least ALERT_COOLDOWN_SECONDS (600 s by default) have passed since the previous message of the same rule; otherwise an ALERT_SUPPRESSED event is logged once. While the value stays above the threshold, a REMINDER is sent every ALERT_REMINDER_SECONDS (600 s). The episode ends with a RESOLVED message only when the value falls below the threshold minus the hysteresis, so readings that fluctuate around the threshold do not produce a flood of alert and resolved pairs.

Rule evaluation never waits for the network. Messages are placed in a bounded queue (50 jobs) and sent by a background worker using `httpx`. Alerts are delivered to Discord through webhook embeds, coloured by type (red for ALERT, orange for REMINDER, green for RESOLVED, blue for TEST). The Discord notification channel is limited to 20 messages per minute with a sliding window, below the platform limit, and the webhook URL is read only from environment variables rather than being stored directly in the source code.

```
1 RULES = (  
2     # key, telemetry field, threshold column, hysteresis, strict (>), title, unit  
3     AlertRule("POLLUTION_HIGH", "gas_ppm", "alert_ppm_threshold",  
4               50.0, True, "POLLUTION", " ppm"),  
5     AlertRule("TEMPERATURE_HIGH", "temperature", "alert_temp_threshold",  
6               2.0, True, "HIGH TEMPERATURE", " °C"),  
7 )  
8  
9 async def evaluate(self, device_id: str, t: TelemetryIn,  
10                   sample_time: datetime) -> None:  
11     thresholds = self._thresholds.get(device_id, DEFAULT_THRESHOLDS)  
12     now = time.monotonic()  
13     for rule in RULES:  
14         value = getattr(t, rule.field)  
15         if value is None:  
16             continue  
17         threshold = thresholds[rule.threshold_key]  
18         over = value > threshold if rule.strict else value >= threshold  
19         st = self._states.setdefault((device_id, rule.key), RuleState())  
20
```

```

21     # (1) Active episode fell below the safe level -> close it
22     if st.active and value < threshold - rule.hysteresis:
23         if st.notified:
24             await self._enqueue(device_id, "RESOLVED", rule, t,
25                                 sample_time, threshold, st.since)
26             st.active = st.notified = st.suppressed_logged = False
27             st.since = None
28             continue
29     # (2) A new episode starts
30     if over and not st.active:
31         st.active, st.notified = True, False
32         st.suppressed_logged, st.since = False, sample_time
33     if not (st.active and over):
34         continue
35     # (3) Not notified yet -> send once the cooldown has passed
36     #     (otherwise an ALERT_SUPPRESSED event is logged once)
37     if not st.notified:
38         if now - st.last_sent >= self.s.alert_cooldown_s:
39             await self._enqueue(device_id, "ALERT", rule, t,
40                                 sample_time, threshold, st.since)
41             st.notified, st.last_sent = True, now
42     # (4) Still above the threshold -> remind every ALERT_REMINDER_SECONDS
43     elif now - st.last_sent >= self.s.alert_reminder_s:
44         await self._enqueue(device_id, "REMINDER", rule, t,
45                             sample_time, threshold, st.since)
46         st.last_sent = now

```

Listing 4.5: Alert rules and episode state machine (backend/app/alert_service.py, excerpt, reformatted, comments translated)

4.7 AI Time-Series Forecasting Model Integration

The forecasting model is developed by Member 5 with TensorFlow/Keras (ai/tv5/train_ai.py). It is an LSTM network that reads a sliding window of 30 consecutive telemetry samples, about 60 seconds at one sample every 2 seconds, with five min-max scaled features: temperature, humidity, gas_ppm and the first differences gas_diff and temp_diff. Two LSTM layers of 64 and 32 units, each followed by dropout 0.2, feed a dense layer of 16 ReLU units and a three-class softmax output: the risk class expected 15 minutes later, SAFE, WARNING or DANGER. The network is trained with sparse categorical cross-entropy and the Adam optimiser on a dataset of 13,000 samples whose labels (target_risk_15m) are prepared in advance.

The trained model (kitchen_lstm_15m.h5) is served by a separate FastAPI container (ai, port 9100, not exposed to the Internet) that exposes /predict, /health and /reload.

The service receives raw samples, computes `gas_diff` and `temp_diff` between adjacent samples and applies the same min-max ranges as the training data before it builds the 30×5 input tensor. A failure of the AI service does not affect monitoring, control or alerting.

Every `AI_INTERVAL_SECONDS` (10 s), the prediction service of the backend:

1. reads the 30 newest telemetry rows of the device and checks the window (Listing 4.6): the newest sample must have reached the backend less than 15 s ago, no sample may miss a reading, and consecutive samples may not be more than 6 s apart. Otherwise the round is skipped and the reason appears in the `detail` field of `GET /prediction`. Without the freshness check, an offline device or a lagging ESP32 clock would make the model forecast the same old window again and again;
2. sends the window to `/predict` and receives the class, its confidence and the probabilities of the three classes;
3. publishes the forecast on `iot/kitchen/{device_id}/ai_prediction` (QoS 0) for the dashboard (Listing 4.7);
4. stores it in `ai_forecasts` with `target_time = time + 15 minutes`; `GET /prediction` returns the latest forecast and `GET /prediction/history` the stored ones;
5. writes an `AI_FORECAST` event only when the predicted class changes, so the event log is not flooded with one entry every 10 seconds.

The LSTM predicts a class, not a gas concentration. For compatibility with an earlier dashboard version, the published message also contains `predicted_gas_ppm`, `predicted_pollution_percent` and `trend`. These three fields are derived from the predicted class with fixed thresholds rather than produced by the network, so they are not stored and are not reported as forecasts. `GET /prediction/accuracy` still evaluates the former regression model; measuring the accuracy of the LSTM on real data requires comparing each stored class with the class observed at its target time, using the labelling rule of the training set.

4.8 Containerized Deployment with Docker Compose

The whole backend is described in `docker-compose.yml` (Table 4.7). The database, the broker, the backend and the AI service all have health checks, and the backend starts only after the other three report healthy.

```

1 # rows = the 30 newest telemetry rows of the device, newest first
2 age = (datetime.now(timezone.utc) - rows[0]["received_at"]).total_seconds()
3 if age > STALE_WINDOW_SECONDS:                                # 15 s
4     return self._skip(f"AI data is stale: newest sample arrived {age:.0f}s ago "
5                       "(device OFFLINE, DB write error or ESP32 clock behind)")
6
7 rows = list(reversed(rows))                                   # chronological order for the LSTM
8
9 # Never skip a broken sample silently: that would change the time sequence
10 for index, row in enumerate(rows):
11     for field in ("temperature", "humidity", "gas_ppm"):
12         if row[field] is None:
13             return self._skip(f"sample {index + 1}/{TIME_STEPS} is missing {field}")
14
15 # Continuity: samples arrive every ~2 s, so a gap above 6 s breaks the window
16 max_gap = timedelta(seconds=MODEL_SAMPLE_SECONDS) * MAX_GAP_FACTOR
17 for previous, current in zip(rows, rows[1:]):
18     delta = current["time"] - previous["time"]
19     if delta.total_seconds() <= 0:
20         return self._skip("invalid telemetry timestamp order")
21     if delta > max_gap:
22         return self._skip(f"AI data has a gap of {delta.total_seconds():.1f}s")

```

Listing 4.6: Checks on the input window before each forecast (prediction_service.py, excerpt, messages translated)

```

1 {
2     "device_id": "esp32_kitchen_01",
3     "timestamp": "2026-09-28T11:20:34.045Z",
4     "target_time": "2026-09-28T11:35:34.045Z",
5     "prediction_kind": "future_risk_classification",
6     "prediction_window_minutes": 15,
7     "hazard_level": 0,
8     "status": "SAFE",
9     "confidence": 0.9618,
10    "probabilities": {"SAFE": 0.9618, "WARNING": 0.0231, "DANGER": 0.0151},
11    "current_gas_ppm": 612.0,
12    "temperature": 30.3,
13    "humidity": 65.1,
14    "recommended_action": "KEEP_AUTO",
15    "model_version": "LSTM15m-ed925e71",
16    "inference_ms": 59.0
17 }

```

Listing 4.7: Main fields of a forecast published on ai_prediction (example values)

Table 4.7: Docker Compose services

Service	Image / build	Ports on the server	Role
timescaledb	timescale/timescaledb:latest-pg17	127.0.0.1:5432	Time-series database
mosquitto	eclipse-mosquitto:2	1883 (public), 127.0.0.1:9001	MQTT broker
backend	Built from ./backend (Python 3.12)	127.0.0.1:8000	Ingestion, REST API, alerts, AI loop
ai	Built from ./ai (Python 3.12, TensorFlow)	127.0.0.1:9100	LSTM model serving
caddy	caddy:2-alpine (production only)	80, 443	HTTPS reverse proxy and static dashboard
simulator	Backend image, profile sim	–	ESP32 simulator for tests without hardware

Three Compose files are layered: the base file, `deploy/docker-compose.prod.yml`, which adds Caddy and `restart: always`, and `deploy/docker-compose.lowmem.yml`, which sets memory limits for a 1 GB virtual machine. On the server the three files are combined by the shell alias `dc` that `deploy/vm-setup.sh` creates, so the whole production stack is started with `dc up -d -build`.

The production server is an AWS EC2 `t3.micro` instance (2 vCPU, 1 GB RAM) running Ubuntu Server 24.04 LTS, with a 30 GB gp3 volume, 4 GB of swap and an Elastic IP address mapped to the domain `iot-kitchen-hcmute.duckdns.org`. The security group allows only four inbound ports: 22 (SSH, administrator IP only), 80 (Let's Encrypt HTTP challenge), 443 (dashboard and API) and 1883 (ESP32). All other ports are bound to `127.0.0.1` through `.env`, so the database, the API, the WebSocket listener and the AI service are reachable only through Caddy or from inside the server. Caddy obtains and renews the TLS certificate automatically (Listing 4.8).

The AI container is the heaviest service: with TensorFlow loaded it uses about 730 MB of memory, while the low-memory profile limits it to 400 MB. It still runs because Docker lets a container use as much swap as its memory limit when no separate swap limit is set, so part of the AI service is kept in the 4 GB swap file. Converting the same network to TensorFlow Lite and running it with the LiteRT interpreter was tested as an optimisation: about 52 MB of memory and 0.2 ms per inference, with outputs matching the original model (maximum probability difference 1.5×10^{-7}).

```

1 {${SITE_DOMAIN}:localhost} {
2     encode gzip
3     handle /mqtt* {

```

```

4     reverse_proxy mosquitto:9001      # MQTT over WebSocket (wss://)
5 }
6 handle /api/* {
7     reverse_proxy backend:8000
8 }
9 handle /docs* {
10    reverse_proxy backend:8000
11 }
12 handle /openapi.json {
13    reverse_proxy backend:8000
14 }
15 handle /health {
16    reverse_proxy backend:8000
17 }
18 handle {
19     root * /srv/web
20     try_files {path} /index.html      # React Router: no 404 on page reload
21     file_server
22 }
23 log {
24     output file /var/log/caddy/access.log
25     format json
26 }
27 }

```

Listing 4.8: Caddy reverse proxy configuration (deploy/Caddyfile)

4.9 Backend – Dashboard Integration Points

Although Chapter 4 focuses on the backend and database infrastructure (the primary domain of Member 3), Member 4, who is responsible for the Web Dashboard UI/UX and data freshness, collaborated directly on the following integration points:

Data contract co-design: topic names, JSON field names (`snake_case`), units, QoS and retain flags are fixed in `docs/data-contract.md` (version 1.2), together with the fields the dashboard should display, such as `seq`, `config_in_sync` and `device_config`.

Two connection paths: MQTT over WebSocket with the read-only `web_dashboard` account delivers the 2-second live values and the ONLINE/OFFLINE status, while the REST API serves history, control and configuration. This avoids polling `/status` every second and still gives access to past data.

Data freshness indicator: the dashboard computes the age of the last message as the current time minus its `timestamp`, using a timer that runs independently of

message arrival. Below 3 s the badge is green, between 3 and 10 s yellow, and above 10 s red, the same levels as `/status`.

Charts: history is requested with `bucket_seconds` (30 s for one hour, 180 s for six hours, 60 s for 24 hours), so the server aggregates the data instead of sending tens of thousands of points to the browser.

Manual override handshake: the dashboard never publishes commands, since the ACL blocks it. The AUTO/MANUAL toggle and the fan switch call `POST /mode` and `POST /actuator`; in production these requests carry the `X-API-Key` header. The response fields `confirmed`, `rtt_ms` and `warning` (returned when the device is still in AUTO mode) tell the user whether the command actually reached the fan.

AI forecast display: forecasts reach the AI tab on `ai_prediction`; `GET /prediction` provides the latest stored forecast when the page opens, and its `available` and `detail` fields tell the user when forecasting is paused. The `class` and its `confidence` are the values to display.

Deployment: the dashboard build is served by Caddy on the same domain as the API and the WebSocket endpoint (`wss://iot-kitchen-hcmute.duckdns.org/mqtt`), so the production site needs no cross-origin requests.

Chapter 5

Real-Time Web Dashboard

5.1 Frontend Architecture and Design Philosophy

In this project, our team engineered an industrial-grade, single-page supervisory control and data acquisition (SCADA) dashboard to monitor indoor air quality and actuate the kitchen ventilation system in real time. Rather than relying on simple, character-limited LCD 1602 screens, we implemented a modern, fully responsive web application capable of high-throughput telemetry ingestion and low-latency interaction.

5.1.1 Technology Stack Selection

The frontend architecture was implemented using modern web development standards:

React 18 (Vite Bundler): Chosen for its virtual DOM diffing efficiency and reactive state management, enabling UI re-renders at sub-16ms intervals without blocking the main event thread. Vite provides near-instantaneous Hot Module Replacement (HMR) and optimized Rollup-based production chunking.

Tailwind CSS: Employed for utility-first styling to create an Obsidian Dark and Cyberpunk Neon Glassmorphism theme, maximizing contrast and readability under challenging kitchen lighting conditions.

Chart.js (v4): Selected for its hardware-accelerated HTML5 Canvas rendering engine, ensuring continuous, lag-free linear interpolation across multi-metric streams.

MQTT.js (Client): A pure JavaScript client library establishing secure WebSocket connections (`wss://`) to our cloud-hosted MQTT Broker.

5.1.2 Component Hierarchy and Responsive Layout

The frontend architecture follows a centralized reactive state model where the top-level **App** component manages hardware telemetry, broker socket lifecycles, and auditory feedback pipelines.



Figure 5.1: Desktop cockpit view showing the command bar, central atmosphere reactor, telemetry matrix, and navigation tabs.

The visual workspace is organized into five core functional blocks:

1. **Command Bar (Header):** Houses the device ID chip (`esp32_kitchen_01`), the packet sequence chip (`seq`), the WiFi diagnostics pill, the audio mute toggle, the CSV data exporter, the broker link status (`ESP32 Online/Offline`), the data freshness pill (e.g., `Live 0.2s`), the threshold settings button, and the Sign Out button.
2. **Central Atmosphere Reactor:** Displays the macro environmental verdict, an indoor health index (0–100), and the current FSM state indicator (`GOOD`, `MID`, `BAD`).
3. **Telemetry Matrix:** Four telemetry cards rendering Room Temperature ($^{\circ}\text{C}$), Relative Humidity (%), MQ-135 Gas Concentration (ppm), and the Turbine Fan actuator with its `AUTO/MANUAL` control mode and `OFF/ON` buttons.
4. **Navigation Tabs:** Seamless view-switching between Live Dynamics & 24H History, Event History Log, and AI Atmospheric Insight.
5. **Continuous Waveform Canvas:** An interactive multi-axis line chart, located below the tabs, rendering sensor histories against the configured FSM threshold lines (Figure 5.2c).

To ensure operators can supervise the kitchen environment from handheld devices while cooking, the dashboard was designed with a mobile-first responsive layout. Figure 5.2 illustrates the three sequential viewports of the mobile interface, showing the stacked structural flow from the header down to the real-time telemetry streams.

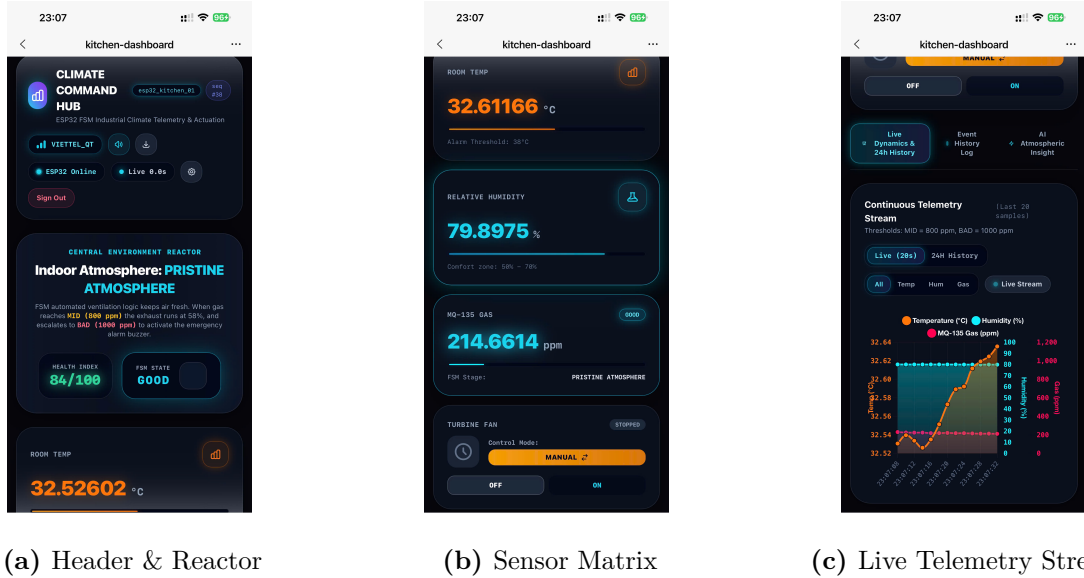


Figure 5.2: Responsive mobile layout captured across 3 vertical viewports for handheld supervision.

5.2 Real-Time MQTT Communication over WebSockets

5.2.1 Event-Driven WebSocket Protocol

Standard HTTP REST polling introduces unacceptable network overhead and server congestion when polling at sub-second frequencies. To achieve true industrial telemetry ingestion, we integrated MQTT over secure WebSockets (`wss://`) via port 9001. Upon user authentication, the client establishes a persistent full-duplex socket with our AWS-hosted Mosquitto broker:

```

1 const clientId =
2   'web_' + Math.random().toString(16).substring(2, 9);
3 const client = mqtt.connect(Ws_URL, {
4   username: WS_USER,
5   password: WS_PASS,
6   clientId,
7   clean: true,
8   reconnectPeriod: 3000,
9   connectTimeout: 5000,
10 });
11 mqttClientRef.current = client;
12
13 client.on('connect', () => {
14   setBrokerConnected(true);
15   client.subscribe('iot/kitchen/esp32_kitchen_01/#');
16 });

```

Listing 5.1: MQTT WebSocket Client Connection Logic

5.2.2 Ghost Packet Suppression and Multi-Layer Filtering

During initial prototype evaluation, an architectural anomaly surfaced: when the physical ESP32 was completely unplugged, the web dashboard would still falsely report the device as “Online” and maintain outdated sensor readings.

Through network packet inspection, we identified two causes:

1. The AWS MQTT Broker persisted the last message published with a `retain = true` flag and immediately flushed this ghost packet whenever the web client connected.
2. Subscribing to the wildcard topic `iot/kitchen/esp32_kitchen_01/#` caused internal backend broadcast events (such as AI predictions and mode updates) to be mistakenly interpreted as active hardware telemetry.

To resolve this issue, we engineered a three-layer packet validation filter directly in the WebSocket receiver callback:

1. **Retained Packet Rejection:** Explicitly discards any packet where `packet.retain == true`.
2. **Topic Isolation:** Ensures that only packets terminating in `/telemetry` reset the hardware watchdog timer.
3. **Monotonic Sequence Verification:** The ESP32 firmware increments an integer sequence counter (`seq++`) with each transmission. If an incoming packet’s `seq` is less than or equal to the cached sequence number, it is discarded as a stale duplicate.

```

1 client.on('message', (topic, message, packet) => {
2   try {
3     const payload = JSON.parse(message.toString());
4     const now = Date.now();
5
6     // Layer 1: reject retained (ghost) packets
7     if (packet && packet.retain) {
8       return;
9     }
10
11    // Layer 2: only /telemetry topics feed the watchdog
12    const isTelemetryTopic = topic.endsWith('/telemetry');
13    if (!isTelemetryTopic) {
14      if (payload.mode) {
15        setTelemetry((prev) => ({ ...prev, mode: payload.mode }));
16      }
17      return;
18    }
19
20    // Layer 3: monotonic sequence check
21    const incomingSeq = Number(payload.seq);
22    if (!isNaN(incomingSeq)) {
23      if (lastValidSeqRef.current !== -1 &&
24        incomingSeq <= lastValidSeqRef.current) {
25        return;
26      }
27      lastValidSeqRef.current = incomingSeq;
28    }
29
30    lastValidPacketTimeRef.current = now;
31    setHasReceivedLivePacket(true);
32    setIsDeviceOffline(false);
33    setTelemetry((prev) => ({ ...prev, ...payload }));
34  } catch (err) {
35    console.error('Error parsing message', err);
36  }
37 });

```

Listing 5.2: Three-Layer MQTT Packet Validation Filter

5.3 Telemetry Widgets and Diagnostic Modals

5.3.1 Telemetry Cards and State Indicators

The four primary metric cards utilize color-coded neon borders and background glows to inform the operator of current conditions:

Room Temperature: Rendered with Flame Neon (`#ff7700`) accompanied by a dynamic progress bar mapped against the 38°C alarm threshold (32.39°C recorded in ambient baseline).

Relative Humidity: Highlighted with Cyan Laser Neon (`#00f0ff`). The card displays the standard 50% – 70% human comfort zone as a reference (80.36% recorded); note that the health score penalty in Section 5.3 uses the wider 40% – 75% band.

MQ-135 Gas Concentration: A reactive multi-state widget dynamically switching between Electric Cyan (GOOD: < 800 ppm), Warning Amber (MID: $800 - 1000$ ppm), and Critical Magenta (BAD: ≥ 1000 ppm) (308.24 ppm recorded in clean air).

Turbine Fan Actuator: Renders a rotating turbine fan icon whose animation correlates directly with actuator state (0.6s rotation when active, stationary when stopped), a status pill (STOPPED / RUNNING (100%)), an AUTO/MANUAL control mode toggle, and OFF/ON buttons.

5.3.2 WiFi Node Diagnostics Modal

Clicking on the WiFi status pill in the header opens a dedicated diagnostics modal (Figure 5.3), displaying the connected SSID (VIETTEL_QT), received signal strength indicator (-58 dBm), assigned IP address (192.168.1.105), and active hardware transmission link (ESP32 TRANSMITTING). A RE-SCAN HARDWARE button refreshes these values.

5.3.3 Air Quality Index and Health Score Formula

To provide occupants with an aggregated safety metric, we implemented an algorithmic Health Score ($S_H \in [0, 100]$):

$$S_H = 100 - P_{\text{gas}} - P_{\text{temp}} - P_{\text{hum}} \quad (5.1)$$



Figure 5.3: WiFi Node Diagnostics Modal providing network performance metrics.

where penalty deductions are calculated based on sensor deviations:

$$P_{\text{gas}} = \min \left(60, \frac{\max(0, \text{PPM} - 400)}{1000} \times 60 \right) \quad (5.2)$$

$$P_{\text{temp}} = \min(20, \max(0, T - 32) \times 2.5) \quad (5.3)$$

$$P_{\text{hum}} = \begin{cases} 15 & \text{if } H < 40\% \text{ or } H > 75\% \\ 0 & \text{otherwise} \end{cases} \quad (5.4)$$

In baseline conditions (308.24 ppm, 32.39°C, 80.36%), the gas penalty is 0 (below 400 ppm), the temperature penalty is approximately 1, and the humidity penalty is 15 (since $H > 75\%$), yielding an aggregate score of 84/100. When the physical hardware enters an offline state, the Health Score is not computed and is displayed as null (-/100) to prevent misleading readings.

5.4 Data Freshness and Dual-Tier Hardware Watchdog

5.4.1 Mathematical Formulation of Data Freshness

Data freshness (Δt_{fresh}) quantifies the transmission latency between the physical microcontroller and the cloud dashboard:

$$\Delta t_{\text{fresh}} = t_{\text{client}} - t_{\text{packet}} \quad (5.5)$$

where t_{client} represents the browser's system clock (`Date.now()`) and t_{packet} is the arrival time of the last verified telemetry packet.

Table 5.1: Data Freshness Classification Spectrum

Latency (Δt_{fresh})	Visual Indicator	Operational Assessment
$\Delta t < 4000$ ms	Cyan Glowing Pulse	Optimal real-time stream (2 s telemetry cycle)
$4000 \leq \Delta t \leq 10000$ ms	Solid Amber	Network jitter or cellular re-routing latency
$\Delta t > 10000$ ms	Pulsing Crimson Red	Hardware link severed / node power loss

5.4.2 Dual-Tier Watchdog Engine (5s Startup vs. 10s Power Cut)

In laboratory testing, we observed that a single timeout threshold failed to handle both cold starts and sudden power cuts effectively. If the dashboard is loaded while the ESP32 is powered off, waiting 60 seconds is impractical. Conversely, during steady-state operation, the system must tolerate minor network anomalies before triggering a full alarm.

We engineered a dual-tier watchdog timer executing on a high-frequency 250ms interval:

1. **Tier 1 (Cold Start Watchdog – 5 Seconds):** Upon initial dashboard mounting, if no valid telemetry packet with an advancing sequence number is registered within 5000ms ($t_{\text{now}} - t_{\text{mount}} \geq 5\text{s}$), the system trips the `isDeviceOffline` flag.
2. **Tier 2 (Runtime Power-Cut Watchdog – 10 Seconds):** If the system was receiving valid data and the ESP32 is abruptly powered off, the watchdog detects that $t_{\text{now}} - t_{\text{last_packet}} \geq 10000\text{ms}$, immediately disconnecting the node.

When `isDeviceOffline` evaluates to `true` (Figure 5.4):

The MQTT status badge transitions to **ESP32 Offline**, and the freshness pill shows the elapsed offline time (e.g., **Offline (7s)**).

The WiFi network pill displays **WiFi Disconnected**.

Telemetry card values are nullified to dashes (-) to eliminate stale sensor illusions, and the health index shows **-/100**.

An emergency banner appears: **ESP32 OFFLINE / DISCONNECTED**, together with a **Re-Scan Signal** button.

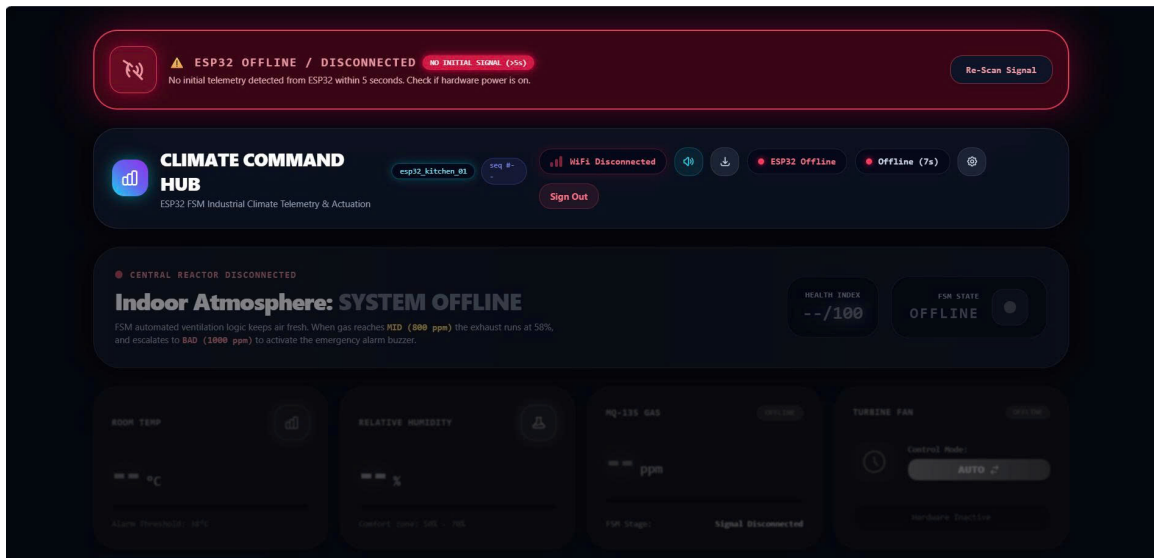


Figure 5.4: Visual state during hardware disconnect: crimson offline banner, dimmed telemetry cards, and nullified metric values (-).

```

1 useEffect(() => {
2   if (!isAuthenticated) return;
3   mountTimeRef.current = Date.now();
4
5   const watchdogTimer = setInterval(() => {
6     const now = Date.now();
7
8     if (lastValidPacketTimeRef.current !== null) {
9       const elapsedSinceLastPacket =
10         now - lastValidPacketTimeRef.current;
11       setFreshnessMs(elapsedSinceLastPacket);
12
13       if (elapsedSinceLastPacket >= RUNTIME_DISCONNECT_TIMEOUT_MS) {
14         setIsDeviceOffline(true);
15       } else {
16         setIsDeviceOffline(false);
17       }
18     } else {
19       const elapsedSinceMount = now - mountTimeRef.current;
20       setFreshnessMs(elapsedSinceMount);
21
22       if (elapsedSinceMount >= INITIAL_STARTUP_TIMEOUT_MS) {
23         setIsDeviceOffline(true);
24       } else {
25         setIsDeviceOffline(false);
26       }
27     }
28   }, 250);
29

```

```

30 return () => clearInterval(watchdogTimer);
31 }, [isAuthenticated]);

```

Listing 5.3: Dual-Tier Watchdog Engine Implementation

5.5 Actuation Control and Decoupled Siren Logic

5.5.1 Bimodal Arbitration: AUTO versus MANUAL

The kitchen climate controller features two operating paradigms:

AUTO Mode: The ESP32’s internal Finite State Machine (FSM) autonomously controls the fan using multi-variable hysteresis thresholds ($P_{\text{mid}} = 800$ ppm, $P_{\text{bad}} = 1000$ ppm). Manual buttons are locked out in the UI (the card displays **Managed by ESP32 FSM Logic**) to prevent operator interference.

MANUAL Mode: The autonomous FSM is paused. The fan responds exclusively to operator commands dispatched from the web interface.

To verify actuator behavior across operational modes, Figure 5.5 highlights the three core control states: manual deactivation (Stopped), manual activation (Running at 100% duty cycle), and autonomous FSM management.

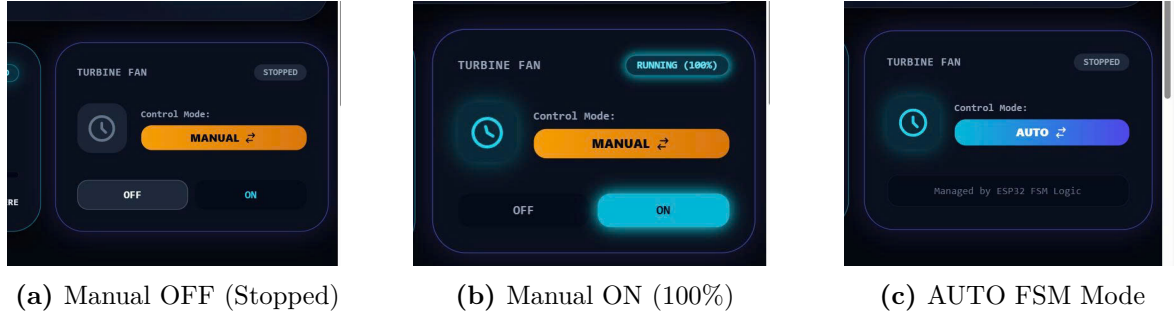


Figure 5.5: Turbine Fan actuator control states: Manual OFF, Manual ON (100%), and AUTO FSM Management.

5.5.2 Binary Actuation and Alarm Decoupling

During prototype evaluation, an unintended interaction occurred: whenever the operator switched to MANUAL mode and activated the fan, the dashboard immediately triggered an emergency alarm banner and sounded the audible buzzer, despite air quality being completely safe ($\text{PPM} \approx 308$).

This issue stemmed from two factors:

1. The ESP32 firmware only implemented binary states (`FAN_GOOD = 0` and `FAN_BAD = 255`). When commanded to activate, it energized the fan to 100% duty cycle. The initial inrush current created a temporary voltage dip on the shared 5V rail, producing an analog reading spike on the MQ-135 sensor.
2. The dashboard UI evaluated `fan_state == 1` as equivalent to a hazard condition, triggering the buzzer.

To rectify this, we simplified the control interface to a clean binary **OFF / ON** mechanism and revised the evaluation logic: manual fan activation is treated as deliberate ventilation rather than an emergency. The siren now triggers strictly when actual gas or temperature readings breach safety limits ($\text{PPM} \geq 1000$ or $T \geq 38^\circ\text{C}$):

```

1 const evalAirQuality = (tel, ppmVal) => {
2   if (isDeviceOffline) {
3     return {
4       badge: 'OFFLINE',
5       levelText: 'SYSTEM OFFLINE',
6       neonColor: '#475569',
7       border: 'border-slate-800 shadow-none',
8       text: 'text-slate-500',
9       badgeBg: 'bg-slate-800 text-slate-500 border-slate-700',
10      isAlarm: false,
11    };
12  }
13
14  const isGasCritical = ppmVal >= configForm.ppm_bad_threshold;
15  const isTempCritical =
16    Number(tel.temperature) >= configForm.temp_threshold &&
17    Number(tel.temperature) > 0;
18  const isHazard = isGasCritical || isTempCritical;
19
20  if (isHazard) {
21    return {
22      badge: 'BAD',
23      levelText: 'CRITICAL HAZARD',
24      neonColor: '#ff0055',
25      border: 'border-rose-500/80 shadow-[0_0_35px_rgba(255,0,85,0.4)]',
26      text: 'text-rose-400',
27      badgeBg: 'bg-rose-500/20 text-rose-300 border-rose-500/60 shadow-[0_0_15px_rgba
    ↪ (255,0,85,0.5)]',
28      isAlarm: true,
29    };
30  }
31
32  if (ppmVal >= configForm.ppm_mid_threshold) {
33    return {

```

```

34     badge: 'MID',
35     levelText: 'MODERATE POLLUTION',
36     neonColor: '#f59e0b',
37     border: 'border-amber-500/70 shadow-[0_0_30px_rgba(245,158,11,0.3)]',
38     text: 'text-amber-400',
39     badgeBg: 'bg-amber-500/20 text-amber-300 border-amber-500/60 shadow-[0
↪ _0_15px_rgba(245,158,11,0.4)]',
40     isAlarm: false,
41 };
42 }
43
44 if (tel.mode === 'MANUAL' && (tel.fan_state === 1 || tel.fan_duty > 0)) {
45     return {
46         badge: 'GOOD',
47         levelText: 'MANUAL VENTILATION ACTIVE',
48         neonColor: '#00f0ff',
49         border: 'border-cyan-500/60 shadow-[0_0_30px_rgba(0,240,255,0.25)]',
50         text: 'text-cyan-400',
51         badgeBg: 'bg-cyan-500/20 text-cyan-300 border-cyan-500/50 shadow-[0_0_15px_rgba
↪ (0,240,255,0.35)]',
52         isAlarm: false,
53     };
54 }
55
56 return {
57     badge: 'GOOD',
58     levelText: 'PRISTINE ATMOSPHERE',
59     neonColor: '#00f0ff',
60     border: 'border-cyan-500/60 shadow-[0_0_30px_rgba(0,240,255,0.25)]',
61     text: 'text-cyan-400',
62     badgeBg: 'bg-cyan-500/20 text-cyan-300 border-cyan-500/50 shadow-[0_0_15px_rgba
↪ (0,240,255,0.35)]',
63     isAlarm: false,
64 };
65 };

```

Listing 5.4: Decoupled Air Quality Evaluation Logic

```

1 const handleSetFanState = async (nextState) => {
2   if (telemetry.mode === 'AUTO') {
3     showToast('Switch to MANUAL mode to control fan!', 'warning');
4     return;
5   }
6   playSound('fan');
7   const targetDuty = nextState === 1 ? 255 : 0;
8   const nextPercent = nextState === 1 ? 100 : 0;
9
10  setTelemetry((prev) => ({
11    ...prev,
12    fan_state: nextState,
13    fan_duty: targetDuty,
14    fan_speed_percent: nextPercent,
15  }));
16  showToast(
17    nextState === 1 ? 'Fan turned ON (100%)' : 'Fan turned OFF (0%)',
18    'success'
19  );
20
21  try {
22    const res = await fetch(`${API_URL}/actuator`, {
23      method: 'POST',
24      headers: {
25        'Content-Type': 'application/json',
26        'X-API-Key': API_KEY,
27      },
28      body: JSON.stringify({ state: nextState }),
29    });
30    const data = await res.json();
31    if (!res.ok) {
32      throw new Error(data.reason || 'Server error (' + res.status + ')');
33    }
34    fetchEvents();
35  } catch (err) {
36    showToast(err.message, 'error');
37  }
38 };

```

Listing 5.5: Binary Manual Actuator Handler

5.6 Continuous Waveform Visualization and Historical Buffering

5.6.1 Chart.js Multi-Axis Stream Integration

The telemetry visualization canvas is powered by Chart.js over an HTML5 canvas element. The chart simultaneously monitors three independent physical dimensions using separate vertical axes, with filter buttons (All / Temp / Hum / Gas) to isolate a single series:

y_{Temp} (Left, °C): Auto-scaled to the data range, with orange neon gradient fill.

y_{Percent} (Right, %): Fixed between 0 – 100% for relative humidity.

y_{PPM} (Far Right, ppm): Range of 0 – 1200 ppm in the Live view, extending up to 10,000 ppm in the 24-hour view when high spikes are recorded, with crimson glow fill for MQ-135 pollution tracking.

5.6.2 Dual-Mode Buffering (Live 20-Sample Stream vs. 24-Hour History)

To satisfy both real-time observation and historical auditing requirements, the dashboard supports dual-mode operation:

1. **Live Mode (20 Samples):** Maintains a circular rolling FIFO buffer holding the 20 most recent data packets. Updates execute with zero-latency linear animation.
2. **24-Hour Historical Engine:** Automatically archives samples at 15-second intervals into browser `localStorage` under the key `kitchen_scada_24h_telemetry`, while asynchronously synchronizing with the backend REST endpoint `GET /history?hours=24` (Figure 5.6). If sparse historical points are detected, a linear interpolation bridge prevents waveform rendering collapse.



Figure 5.6: 24-Hour Historical Environmental Record view showing 291 cached records, a max gas peak of 9999 ppm, and 9 hazard detections.

```

1 const appendTo24hHistory = (sample) => {
2   const now = Date.now();
3   if (now - lastSaveTimeRef.current < 15000) return;
4   lastSaveTimeRef.current = now;
5
6   setHistory24h((prev) => {
7     const cutoff = now - 24 * 60 * 60 * 1000;
8     const safePrev = Array.isArray(prev) ? prev : [];
9     const updated = [
10      ...safePrev.filter((p) => p && p.ts >= cutoff),
11      { ts: now, ...sample },
12    ];
13    try {
14      localStorage.setItem(STORAGE_24H_KEY, JSON.stringify(updated));
15    } catch {
16      // Safe quota
17    }
18    return updated;
19  });
20 };

```

Listing 5.6: 24-Hour Historical Telemetry Storage Ingestion

5.6.3 System Event Logging and Threshold Configuration Modals

Beyond continuous telemetry graphs, the dashboard incorporates operational logging and dynamic parameter tuning:

Event History Log: Synchronized every 2.5s (with a manual **Sync Now** button), logging critical state transitions in reverse chronological order (Figure 5.7). Recorded events include fan state changes, mode changes, device online notifications, threshold mismatch warnings, AI model loading, and backend startup. They demonstrate a rapid round-trip time (RTT) of 150.8 ms for manual fan commands (**MANUAL_FAN_COMMAND**) and AI forecast validations (99.13% confidence).

Threshold Tuning Modal: A secured configuration popup allowing operators to calibrate FSM trigger thresholds ($P_{\text{mid}} = 800$ ppm, $P_{\text{bad}} = 1000$ ppm, Alert = 1200 ppm, $T_{\text{threshold}} = 38^{\circ}\text{C}$, Dwell Time = 30s) directly from the browser (Figure 5.8).

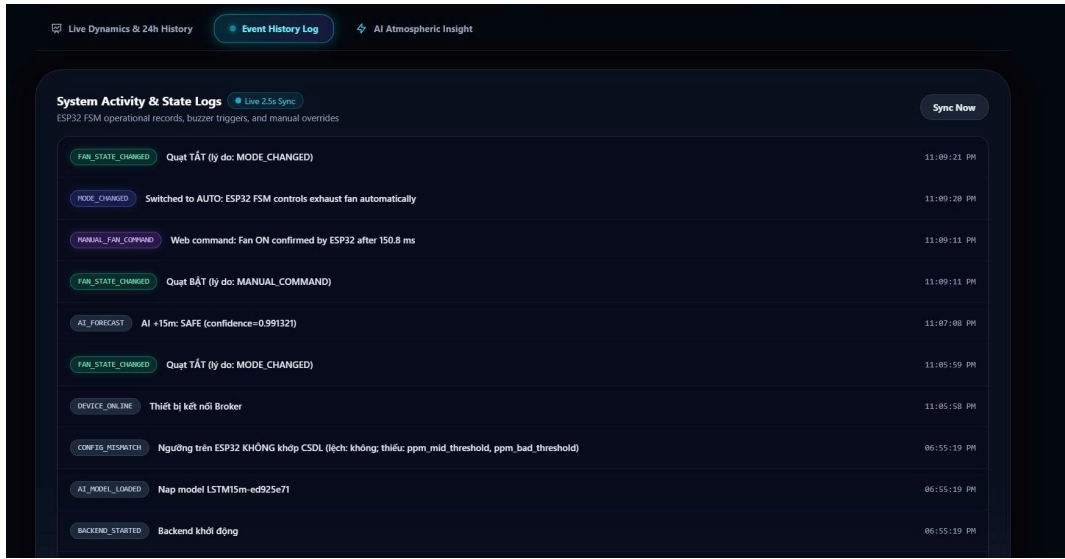


Figure 5.7: System Activity and State Logs view displaying RTT performance (150.8 ms) and AI inferences.

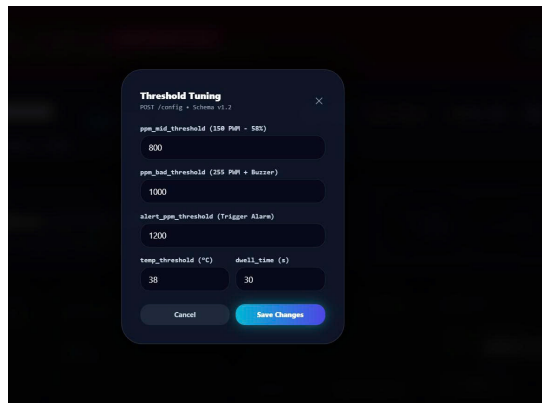


Figure 5.8: Threshold Tuning calibration modal adhering to schema v1.2.

5.6.4 Exporting Telemetry to CSV

For offline data verification and academic submission, operators can export active telemetry buffers directly into standard RFC 4180-compliant CSV files with a single click. The function constructs a data URI blob and triggers a synthetic anchor click, downloading files formatted with timestamp suffixes.

5.7 Testing and Deployment Verification

5.7.1 Production Deployment on Vercel

The web dashboard was packaged and deployed on Vercel's global edge network. During continuous deployment via GitHub Actions, our team resolved single-page application (SPA) routing 404 errors by implementing a rewrite rule within `vercel.json`:

```
1 {  
2   "rewrites": [  
3     {  
4       "source": "/(.*)",  
5       "destination": "/index.html"  
6     }  
7   ]  
8 }
```

Listing 5.7: Vercel SPA Routing Configuration

Through this multi-stage architectural design, our team delivered a reliable real-time supervisory web interface that successfully connects our hardware, cloud broker, and automated ventilation subsystems.

Chapter 6

Theoretical Foundations of Artificial Intelligence, Model Testing and Quantitative Experiments of End-to-End IoT Systems

6.1 Overview of the Indoor Air Quality (IAQ) Prediction Problem

In the context of rapid urbanization, indoor air quality (IAQ) has become a crucial factor directly affecting human health and the energy efficiency of buildings. Unlike the outdoor environment, the indoor microclimate of a kitchen is characterized by abrupt and highly localized variations in pollutants. Cooking and frying activities generate large amounts of particulate matter ($\text{PM}_{2.5}$, PM_{10}), volatile organic compounds (VOCs), and toxic gases such as carbon monoxide (CO). These substances diffuse very quickly in enclosed spaces, especially when combined with high temperatures and humidity generated by the heating element.

A passive ventilation system, activated only when pollutant concentrations exceed safe thresholds, often exhibits physical inertia in the dilution and expulsion of toxic gases. Therefore, a shift from reactive control to proactive control is essential. By integrating Artificial Intelligence (AI), the system is capable of estimating pollution concentrations in the short term (the next 10 to 30 minutes), thereby providing foundational data for the system to generate early activation scenarios, minimizing exposure risks for users.

Predicting pollutant concentrations in a kitchen is not a simple linear extrapolation problem. Smoke accumulation depends on many variables with strong covariance. For

example, an increase in temperature increases the kinetic energy of gas molecules, accelerating diffusion; meanwhile, high humidity directly interferes with the physicochemical properties of semiconductor surfaces (SnO_2 of the MQ-135 sensor), changing the resistance R_s and generating nonlinear noise. Therefore, the machine learning model designed must be a multivariate time series forecasting model.

6.2 Design of Features and Structure of the Training Dataset

To support supervised learning, the system requires a dataset that accurately reflects the dynamic states of airflow in the kitchen space. Data collection is conducted directly on an experimental chamber model ($30 \times 20 \times 25$ cm) designed by Member 1. The simulated scenarios include multiple lifecycles: baseline state (clean air), cooking start phase (exponential increase in concentration), peak pollution phase, forced fan phase (decreased concentration), and residual effect phase (smoke residue).

The algorithm uses a sliding window mechanism to establish a continuous observation sequence. The window size is set to the 20 most recent time samples. Input at time t is a multidimensional matrix, comprising:

Equivalent pollution concentration (P_t): Indicates the concentration level of smoke and VOCs.

Rate of change of concentration (dP/dt): The first derivative of concentration with respect to time, which helps the model identify sudden upward or downward trends (pulse properties).

Ambient temperature (T_t): Thermodynamic characteristics.

Relative humidity (H_t): Noise variables affecting the sensor.

The input state vector is represented as:

$$X = [P_{t-19}, \dots, P_t, dP/dt_{t-19}, \dots, dP/dt_t, T_{t-19}, \dots, T_t, H_{t-19}, \dots, H_t] \quad (6.1)$$

The expected output of the model is a scalar value Y_{t+h} , representing the predicted level of pollution at time $t + h$ (with h being the forecast horizon, set at 15 minutes).

6.3 Mathematical Foundations of Applied Artificial Intelligence Algorithms

The research process involved training, hyperparameter tuning, and cross-comparing two main algorithmic approaches: Holt-Winters Exponential Smoothing and Long Short-Term Memory (LSTM). This choice was based on balancing nonlinear learning accuracy and computational cost on an edge-to-cloud architecture.

6.3.1 Holt-Winters Exponential Smoothing Model (HWES)

Exponential smoothing originates from assigning progressively decreasing weights to past observations, based on the logical principle that the closer the data is to the present, the more valuable information it contains for the future. The Holt-Winters method (Double/Triple Exponential Smoothing) extends this theory by decomposing time series into three independent structural components: Level, Trend, and Seasonality.

In a kitchen environment, pollution fluctuations are not seasonal or fixed long-term cycles (like the annual climate cycle), but rather short-term cycles due to algorithms controlling the on/off switching of the fan (chattering) and the on/off cycles of the induction cooktop. Therefore, the Holt-Winters model is configured with seasonal cycles m , corresponding to the cycle of a typical cooking session.

The multiplicative Holt-Winters smoothing equations describe the system as follows:

Update Base Level (L_t):

$$L_t = \alpha \frac{Y_t}{S_{t-m}} + (1 - \alpha)(L_{t-1} + \phi B_{t-1}) \quad (6.2)$$

Update Trend Slope (B_t):

$$B_t = \beta^*(L_t - L_{t-1}) + (1 - \beta^*)\phi B_{t-1} \quad (6.3)$$

Update Seasonality Index (S_t):

$$S_t = \gamma \frac{Y_t}{L_{t-1} + \phi B_{t-1}} + (1 - \gamma)S_{t-m} \quad (6.4)$$

Forecast h steps into the future ($Y_{t+h|t}$):

$$Y_{t+h|t} = \left(L_t + \sum_{i=1}^h \phi^i B_t \right) S_{t-m+h} \quad (6.5)$$

Here, α, β^*, γ are smoothing parameters within the range $[0, 1]$, which determine the learning rate of the model for new variations. A key improvement in this architecture is the integration of the damping parameter $\phi \in (0, 1)$. When smoke concentrations surge, a conventional linear model would predict pollution concentrations approaching infinity. The Damped Trend technique forces the trend slope to decrease over time, a rough estimate that closely matches the physical reality when the amount of gas produced reaches equilibrium or the concentration disperses to saturation in space.

6.3.2 Short-Term Memory Network (LSTM)

In addition to statistical models, Long Short-Term Memory Networks (LSTMs) are implemented to fully exploit the nonlinear correlations between input features. LSTM is an evolved form of Recurrent Neural Networks (RNNs), specifically designed to address the vanishing gradient phenomenon when processing long-term time series.

At the core of LSTM is Cell State C_t , which acts as a conveyor belt of information running along the entire time series. Interference with this information flow is controlled by Gates, which use a Sigmoid trigger function to decide the percentage of information to be retained (equal to 1) or discarded (equal to 0).

The mathematical matrix governing an LSTM neuron block includes:

Forget Gate: Determines the amount of historical information to be removed.

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad (6.6)$$

Input Gate and Candidate Cell State: Select new information to update.

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad (6.7)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C) \quad (6.8)$$

Update Cell State:

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \quad (6.9)$$

Output Gate and Hidden State:

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad (6.10)$$

$$h_t = o_t \odot \tanh(C_t) \quad (6.11)$$

The LSTM architecture applied to the project is designed as a lightweight architecture, consisting of 2 hidden layers with 32 LSTM units each, followed by fully connected dense

layers to synthesize features and output a continuous prediction value. Although LSTM requires more computational power and has a slower inference time than HWES, its ability to extract complex patterns from the interplay of temperature, humidity, and smoke makes it a strong candidate for the resource-rich backend subsystem on Docker.

6.4 Model Accuracy Assessment: Relationship Between MAE and RMSE

To quantify the error of the machine learning model, the project adheres to rigorous evaluation standards through two statistical indicators: Mean Absolute Error (MAE) and Root Mean Square Error (RMSE).

The simultaneous use of both indicators has profound interpretive significance. Their formulas are defined as:

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (6.12)$$

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \quad (6.13)$$

From the perspective of Bayesian statistics and Maximum Likelihood Estimation (MLE), MAE is equivalent to optimizing the likelihood function assuming the error follows a Laplace distribution, while RMSE is equivalent to optimizing when the error follows a Normal (Gaussian) distribution.

The key difference lies in RMSE performing the squared deviation $(y_i - \hat{y}_i)^2$ before averaging. This mathematical operation transforms RMSE into a highly sensitive measure and severely punishes outliers—i.e., predictions that deviate significantly from reality. In the context of kitchen air risk management, an outlier represents a moment of peak in toxic smoke concentration that the AI model cannot predict. If the model has low MAE but high RMSE, it means the model predicts very well in normal conditions but fails miserably during a sudden smoke incident. Therefore, the ultimate goal of the AI training process in this project is to force the RMSE value down to near the MAE level, demonstrating the model's ability to reliably capture the sharpness of anomalous pollution peaks.

6.5 Quantitative Experimentation and Testing of Industrial Readiness

The success of an end-to-end IoT project lies not only in sophisticated algorithms or schematic design, but also in its ability to operate in practice under harsh conditions. The research team designed a comprehensive Test Matrix comprising four quantitative experimental measurements.

6.5.1 Test 1: Measuring System-Wide Latency (Latency & RTT)

Purpose: This test quantitatively evaluates the response speed of the entire transmission chain, from physical input at the Edge Node to Cloud response, and from control command issuance on the Web Dashboard to physical execution by the actuator.

Experimental Results:

Analytical measure (Metric)	Min (ms)	Max (ms)	Average (ms)	Standard deviation (ms)
Telemetry delay (Sensor → Web)	137.17	224.48	180.30	22.50
RTT latency (Web → Fan → Web)	180.00	309.83	240.88	32.35

Table 6.1: Distribution of telemetry latency and control loop response time across 30 samples

In-depth Perspective: The telemetry stream uses QoS 0 (Fire and Forget), optimizing bandwidth for continuous time-series data delivery with an average latency of 180.30 ms. Conversely, the device control stream applies QoS 1, requiring the MQTT Broker to store and receive acknowledgment from the ESP32, resulting in an RTT of 240.88 ms.

6.5.2 Test 2: Evaluating Chattering Phenomena (On/Off Oscillations) Under the Influence of FSM

Purpose: Demonstrates the superiority of a Finite State Machine (FSM) programmed with Hysteresis (Space Delay Range) and Minimum Dwell Time compared to the purely conditional programming method in eliminating relay or MOSFET chattering.

Control Algorithm Scenario	Number of switches (in 5 mins)	Observable physical state
Linear single threshold (IF-ELSE)	21 times	Constant clicking sound, M FET generates heat, OLED experiences voltage drops and flickering.
FSM Hysteresis & Dwell Time (30s)	2 times	Smooth operation, decisive transitions once the air is clear.

Table 6.2: Comparison of the control stability of physical actuators under amplitude disturbances

6.5.3 Test 3: Local Autonomy in the Event of Network Fallback and Connection Recovery

Purpose: Assesses the survivability of embedded devices when isolated from the network, verifying local self-control and measuring reconnection time.

Results: When the Wi-Fi router was unplugged during a high-smoke scenario, the ESP32 continued to operate the 12V fan independently via local FSM logic. Upon restoring network access, the system reconnected within an average of 12 seconds, utilizing an exponential backoff strategy and transmitting status updates reliably via LWT and QoS 1 queues.

6.5.4 Test 4: Evaluation of Ventilation Performance and Air Clearance Time According to ASHRAE 62.2

Purpose: Quantifies the practical applicability of the model and compares the fluid dynamics of forced air flow with natural air diffusion according to ASHRAE Standard 62.2.

Natural cleaning time (T_1): 15 minutes 35 seconds (935 seconds).

Forced ventilation cleaning time (T_2): 1 minute 12 seconds (72 seconds).

$$\text{Time saving efficiency (\%)} = \frac{T_1 - T_2}{T_1} \times 100\% = \frac{935 - 72}{935} \times 100\% \approx 92.30\% \quad (6.14)$$

In-depth Perspective: Relying on natural ventilation in a humid tropical monsoon climate results in sluggish air circulation. Applying a mechanical exhaust fan system accelerates purification by up to 13 times (reducing emissions by 92.30%) and creates a directed negative pressure airflow structure.

6.6 Project Summary and Practical Recommendations

Chapter 6 comprehensively illustrates the design capabilities, software integration, and experimental demonstrations of the IoT kitchen monitoring and ventilation system. The quantitative experiments confirmed the local survival capabilities of the Edge Device and optimized the time taken to clean toxic gases, demonstrating full compliance with international and regional ventilation and IAQ standards.

Chapter 7

Future Developments

The system has met its technical objectives: the hardware, the MQTT communication, the TimescaleDB time-series database, the SCADA-style web dashboard and the LSTM risk forecast all work together. The design and the tests also point to several directions that would improve performance, scalability and the potential for commercial use.

7.1 Multi-Room and Multi-Zone Deployment

The current prototype is limited to one simulated kitchen chamber. The topic structure `iot/kitchen/{device_id}/<topic>` was designed so that further nodes can be added without changing the topic names. It can therefore be extended to several zones of a smart home (main kitchen, laundry area, living room), so that the backend and the dashboard collect, aggregate and manage many edge nodes at the same time without changes to the core code.

7.2 Edge AI with a More Powerful Microcontroller and TensorFlow Lite

At present the LSTM model is trained offline and runs in a separate container on the cloud server (AWS EC2). To reduce network latency and to keep the forecast available when the Internet connection is lost, the model can be converted to TensorFlow Lite and embedded in a microcontroller with AI acceleration, such as the ESP32-S3 or ESP32-P4. The conversion has already been tested on the server (Chapter 4): the memory use dropped to about 52 MB with outputs matching the original model. Running the inference on the device would let the node forecast the risk on its own, independently of the network.

7.3 Completing the Dwell-Time Lock

As discussed in Chapter 3, the FSM already uses a hysteresis band ($H_P = 50$ ppm) to remove chattering at the BAD threshold. The parameter `dwell_time_seconds` of the data contract v1.2 is, however, only stored and reported; it is not yet used to lock the state. A future version will complete this logic so that each new state is held for a minimum time, which removes the remaining short toggles at the GOOD/MID boundary and protects the fan motor and the MOSFET.

7.4 Industrial-Grade Sensors and Real-World Validation

The MQ-135 used in the prototype is a low-cost semiconductor sensor, suitable for study. It is sensitive, but it is affected by humidity and its characteristics drift with ageing. In commercial applications the system could use non-dispersive infrared (NDIR) sensors for an accurate CO₂ measurement, and it should be calibrated and validated against industrial-grade reference instruments in real kitchens. Such data would also allow the accuracy of the LSTM forecast to be measured on real observations.

Bibliography

- [1] Espressif Systems, *ESP32 Series Datasheet*, Espressif Systems.
- [2] Zhengzhou Winsen Electronics (Hanwei), *MQ-135 Gas Sensor Technical Data*.
- [3] ASAIR / Aosong Electronics, *AHT20 Temperature and Humidity Sensor Datasheet*.
- [4] NXP Semiconductors, *PCF8574/PCF8574A Remote 8-bit I/O Expander for I²C-bus*, datasheet.
- [5] Texas Instruments, *LM2596 SIMPLE SWITCHER 3-A Step-Down Voltage Regulator*, datasheet.
- [6] OASIS, *MQTT Version 3.1.1 Specification*. <https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/>
- [7] Nick O’Leary, *PubSubClient – MQTT client library for Arduino*. <https://pubsubclient.knolleary.net/>
- [8] Benoit Blanchon, *ArduinoJson Documentation*. <https://arduinojson.org/>
- [9] Eclipse Foundation, *Eclipse Mosquitto Documentation*. <https://mosquitto.org/documentation/>
- [10] PostgreSQL Global Development Group, *PostgreSQL Documentation*. <https://www.postgresql.org/docs/>
- [11] Timescale, *TimescaleDB Documentation*. <https://docs.timescale.com/>
- [12] FastAPI Project, *FastAPI Documentation*. <https://fastapi.tiangolo.com/>
- [13] Discord, *Webhook Resource Documentation*. <https://discord.com/developers/docs/resources/webhook>
- [14] Docker, *Docker Compose Documentation*. <https://docs.docker.com/compose/>
- [15] Google, *TensorFlow and Keras Documentation*. <https://www.tensorflow.org/>

- [16] Caddy Project, *Caddy Documentation*. <https://caddyserver.com/docs/>
- [17] Meta Open Source, *React Documentation*. <https://react.dev/>
- [18] Vite Project, *Vite Documentation*. <https://vite.dev/>
- [19] Chart.js Contributors, *Chart.js Documentation*. <https://www.chartjs.org/docs/>
- [20] MQTT.js Contributors, *MQTT.js Documentation*. <https://github.com/mqttjs/MQTT.js>
- [21] Vercel, *Vercel Documentation*. <https://vercel.com/docs>
- [22] Amazon Web Services, *Amazon EC2 Documentation*. <https://docs.aws.amazon.com/ec2/>